DATA MINING COURSE READINGS

Chapter 02: Data Preprocessing

Curated Textbook Readings from Tan et al. · Matching Lecture: IE500_DM_02_Preprocessing.pdf

REFERENCE TEXTBOOK

Introduction to Data Mining (2nd Edition, Global Edition)

Authors: Pang-Ning Tan, Michael Steinbach, Anuj Karpatne, Vipin Kumar | Pearson, 2019

Curated by: Ehsan Shahbazi | Shahid Beheshti University (SBU)

INCLUDED CHAPTER SECTIONS & TOPIC SCOPE

- **Section 2.1: Types of Data** — Book p. 47·62
  Attributes and measurement types, record, matrix, graph, and ordered data
- **Section 2.2: Data Quality** — Book p. 63·70
  Measurement/collection issues, noise, outliers, missing values, duplicate data
- **Section 2.3: Data Preprocessing** — Book p. 71·91
  Aggregation, sampling, PCA dimensionality reduction, feature selection, normalization
- **Section 2.4: Similarity and Dissimilarity** — Book p. 92·120
  Distance metrics (Euclidean, Manhattan), Jaccard, Cosine similarity for documents
- **Section 6.11: Class Imbalance Problem** — Book p. 516·533
  Handling imbalanced datasets, resampling methods, oversampling, SMOTE

STUDENT STUDY GUIDE & SCOPE

Covers fundamental data representations, data quality issues (missing values, noise), feature selection, PCA, sampling strategies, document proximity metrics, and handling imbalanced datasets (SMOTE/resampling).

2

Data

This chapter discusses several data-related issues that are important for successful data mining:

The Type of Data Data sets differ in a number of ways. For example, the attributes used to describe data objects can be of different types—quantitative or qualitative—and data sets often have special characteristics; e.g., some data sets contain time series or objects with explicit relationships to one another. Not surprisingly, the type of data determines which tools and techniques can be used to analyze the data. Indeed, new research in data mining is often driven by the need to accommodate new application areas and their new types of data.

The Quality of the Data Data is often far from perfect. While most data mining techniques can tolerate some level of imperfection in the data, a focus on understanding and improving data quality typically improves the quality of the resulting analysis. Data quality issues that often need to be addressed include the presence of noise and outliers; missing, inconsistent, or duplicate data; and data that is biased or, in some other way, unrepresentative of the phenomenon or population that the data is supposed to describe.

Preprocessing Steps to Make the Data More Suitable for Data Mining Often, the raw data must be processed in order to make it suitable for analysis. While one objective may be to improve data quality, other goals focus on modifying the data so that it better fits a specified data mining technique or tool. For example, a continuous attribute, e.g., length, sometimes needs to be transformed into an attribute with discrete categories, e.g., *short*, *medium*, or *long*, in order to apply a particular technique. As another example, the

number of attributes in a data set is often reduced because many techniques are more effective when the data has a relatively small number of attributes.

Analyzing Data in Terms of Its Relationships One approach to data analysis is to find relationships among the data objects and then perform the remaining analysis using these relationships rather than the data objects themselves. For instance, we can compute the similarity or distance between pairs of objects and then perform the analysis—clustering, classification, or anomaly detection—based on these similarities or distances. There are many such similarity or distance measures, and the proper choice depends on the type of data and the particular application.

Example 2.1 (An Illustration of Data-Related Issues)**.** To further illustrate the importance of these issues, consider the following hypothetical situation. You receive an email from a medical researcher concerning a project that you are eager to work on.

> Hi,
>
> I've attached the data file that I mentioned in my previous email. Each line contains the information for a single patient and consists of five fields. We want to predict the last field using the other fields. I don't have time to provide any more information about the data since I'm going out of town for a couple of days, but hopefully that won't slow you down too much. And if you don't mind, could we meet when I get back to discuss your preliminary results? I might invite a few other members of my team.
>
> Thanks and see you in a couple of days.

Despite some misgivings, you proceed to analyze the data. The first few rows of the file are as follows:

```
012   232   33.5   0    10.7
020   121   16.9   2   210.1
027   165   24.0   0   427.6
⋮
```

A brief look at the data reveals nothing strange. You put your doubts aside and start the analysis. There are only 1000 lines, a smaller data file than you had hoped for, but two days later, you feel that you have made some progress. You arrive for the meeting, and while waiting for others to arrive, you strike

up a conversation with a statistician who is working on the project. When she learns that you have also been analyzing the data from the project, she asks if you would mind giving her a brief overview of your results.

Statistician: So, you got the data for all the patients?

Data Miner: Yes. I haven't had much time for analysis, but I do have a few interesting results.

Statistician: Amazing. There were so many data issues with this set of patients that I couldn't do much.

Data Miner: Oh? I didn't hear about any possible problems.

Statistician: Well, first there is field 5, the variable we want to predict. It's common knowledge among people who analyze this type of data that results are better if you work with the log of the values, but I didn't discover this until later. Was it mentioned to you?

Data Miner: No.

Statistician: But surely you heard about what happened to field 4? It's supposed to be measured on a scale from 1 to 10, with 0 indicating a missing value, but because of a data entry error, all 10's were changed into 0's. Unfortunately, since some of the patients have missing values for this field, it's impossible to say whether a 0 in this field is a real 0 or a 10. Quite a few of the records have that problem.

Data Miner: Interesting. Were there any other problems?

Statistician: Yes, fields 2 and 3 are basically the same, but I assume that you probably noticed that.

Data Miner: Yes, but these fields were only weak predictors of field 5.

Statistician: Anyway, given all those problems, I'm surprised you were able to accomplish anything.

Data Miner: True, but my results are really quite good. Field 1 is a very strong predictor of field 5. I'm surprised that this wasn't noticed before.

Statistician: What? Field 1 is just an identification number.

Data Miner: Nonetheless, my results speak for themselves.

Statistician: Oh, no! I just remembered. We assigned ID numbers after we sorted the records based on field 5. There is a strong connection, but it's meaningless. Sorry.

■

Although this scenario represents an extreme situation, it emphasizes the importance of "knowing your data." To that end, this chapter will address each of the four issues mentioned above, outlining some of the basic challenges and standard approaches.

2.1 Types of Data

A **data set** can often be viewed as a collection of **data objects**. Other names for a data object are *record*, *point*, *vector*, *pattern*, *event*, *case*, *sample*, *instance*, *observation*, or *entity*. In turn, data objects are described by a number of **attributes** that capture the characteristics of an object, such as the mass of a physical object or the time at which an event occurred. Other names for an attribute are *variable*, *characteristic*, *field*, *feature*, or *dimension*.

Example 2.2 (Student Information)**.** Often, a data set is a file, in which the objects are records (or rows) in the file and each field (or column) corresponds to an attribute. For example, Table 2.1 shows a data set that consists of student information. Each row corresponds to a student and each column is an attribute that describes some aspect of a student, such as grade point average (GPA) or identification number (ID).

Table 2.1. A sample data set containing student information.

Student ID	Year	Grade Point Average (GPA)	...
	⋮		
1034262	Senior	3.24	...
1052663	Sophomore	3.51	...
1082246	Freshman	3.62	...
	⋮		

■

Although record-based data sets are common, either in flat files or relational database systems, there are other important types of data sets and systems for storing data. In Section 2.1.2, we will discuss some of the types of data sets that are commonly encountered in data mining. However, we first consider attributes.

2.1.1 Attributes and Measurement

In this section, we consider the types of attributes used to describe data objects. We first define an attribute, then consider what we mean by the type of an attribute, and finally describe the types of attributes that are commonly encountered.

What Is an Attribute?

We start with a more detailed definition of an attribute.

Definition 2.1. An **attribute** is a property or characteristic of an object that can vary, either from one object to another or from one time to another.

For example, eye color varies from person to person, while the temperature of an object varies over time. Note that eye color is a symbolic attribute with a small number of possible values $\{brown, black, blue, green, hazel, etc.\}$, while temperature is a numerical attribute with a potentially unlimited number of values.

At the most basic level, attributes are not about numbers or symbols. However, to discuss and more precisely analyze the characteristics of objects, we assign numbers or symbols to them. To do this in a well-defined way, we need a measurement scale.

Definition 2.2. A **measurement scale** is a rule (function) that associates a numerical or symbolic value with an attribute of an object.

Formally, the process of **measurement** is the application of a measurement scale to associate a value with a particular attribute of a specific object. While this may seem a bit abstract, we engage in the process of measurement all the time. For instance, we step on a bathroom scale to determine our weight, we classify someone as male or female, or we count the number of chairs in a room to see if there will be enough to seat all the people coming to a meeting. In all these cases, the "physical value" of an attribute of an object is mapped to a numerical or symbolic value.

With this background, we can discuss the type of an attribute, a concept that is important in determining if a particular data analysis technique is consistent with a specific type of attribute.

The Type of an Attribute

It is common to refer to the type of an attribute as the **type of a measurement scale**. It should be apparent from the previous discussion that an

attribute can be described using different measurement scales and that the properties of an attribute need not be the same as the properties of the values used to measure it. In other words, the values used to represent an attribute can have properties that are not properties of the attribute itself, and vice versa. This is illustrated with two examples.

Example 2.3 (Employee Age and ID Number)**.** Two attributes that might be associated with an employee are *ID* and *age* (in years). Both of these attributes can be represented as integers. However, while it is reasonable to talk about the average age of an employee, it makes no sense to talk about the average employee ID. Indeed, the only aspect of employees that we want to capture with the ID attribute is that they are distinct. Consequently, the only valid operation for employee IDs is to test whether they are equal. There is no hint of this limitation, however, when integers are used to represent the employee ID attribute. For the age attribute, the properties of the integers used to represent age are very much the properties of the attribute. Even so, the correspondence is not complete because, for example, ages have a maximum, while integers do not. ■

Example 2.4 (Length of Line Segments)**.** Consider Figure 2.1, which shows some objects—line segments—and how the length attribute of these objects can be mapped to numbers in two different ways. Each successive line segment, going from the top to the bottom, is formed by appending the topmost line segment to itself. Thus, the second line segment from the top is formed by appending the topmost line segment to itself twice, the third line segment from the top is formed by appending the topmost line segment to itself three times, and so forth. In a very real (physical) sense, all the line segments are multiples of the first. This fact is captured by the measurements on the right side of the figure, but not by those on the left side. More specifically, the measurement scale on the left side captures only the ordering of the length attribute, while the scale on the right side captures both the ordering and additivity properties. Thus, an attribute can be measured in a way that does not capture all the properties of the attribute. ■

Knowing the type of an attribute is important because it tells us which properties of the measured values are consistent with the underlying properties of the attribute, and therefore, it allows us to avoid foolish actions, such as computing the average employee ID.

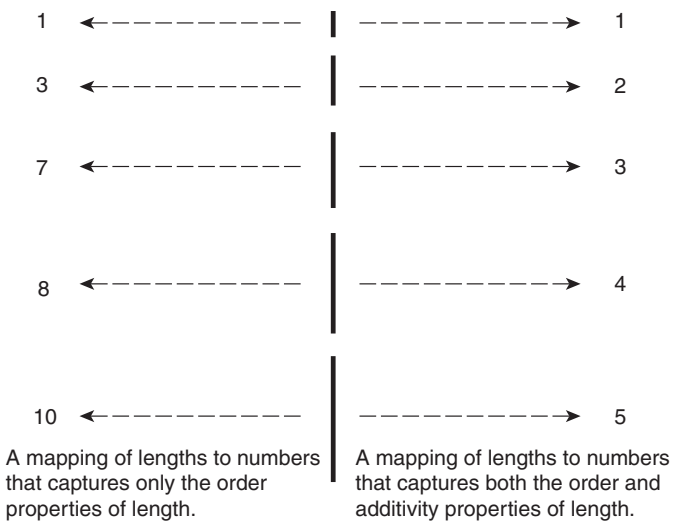


Figure 2.1. The measurement of the length of line segments on two different scales of measurement.

The Different Types of Attributes

A useful (and simple) way to specify the type of an attribute is to identify the properties of numbers that correspond to underlying properties of the attribute. For example, an attribute such as length has many of the properties of numbers. It makes sense to compare and order objects by length, as well as to talk about the differences and ratios of length. The following properties (operations) of numbers are typically used to describe attributes.

1. **Distinctness** $=$ and $\neq$
2. **Order** $<$, $\leq$, $>$, and $\geq$
3. **Addition** $+$ and $-$
4. **Multiplication** $\times$ and $/$

Given these properties, we can define four types of attributes: **nominal**, **ordinal**, **interval**, and **ratio**. Table 2.2 gives the definitions of these types, along with information about the statistical operations that are valid for each type. Each attribute type possesses all of the properties and operations of the attribute types above it. Consequently, any property or operation that is valid for nominal, ordinal, and interval attributes is also valid for ratio attributes. In other words, the definition of the attribute types is cumulative. However,

Table 2.2. Different attribute types.

Attribute Type		Description	Examples	Operations
Categorical (Qualitative)	Nominal	The values of a nominal attribute are just different names; i.e., nominal values provide only enough information to distinguish one object from another. $(=, \neq)$	zip codes, employee ID numbers, eye color, gender	mode, entropy, contingency correlation, χ^2 test
	Ordinal	The values of an ordinal attribute provide enough information to order objects. $(<, >)$	hardness of minerals, $\{good, better, best\}$, grades, street numbers	median, percentiles, rank correlation, run tests, sign tests
Numeric (Quantitative)	Interval	For interval attributes, the differences between values are meaningful, i.e., a unit of measurement exists. $(+, -)$	calendar dates, temperature in Celsius or Fahrenheit	mean, standard deviation, Pearson's correlation, t and F tests
	Ratio	For ratio variables, both differences and ratios are meaningful. $(\times, /)$	temperature in Kelvin, monetary quantities, counts, age, mass, length, electrical current	geometric mean, harmonic mean, percent variation

this does not mean that the statistical operations appropriate for one attribute type are appropriate for the attribute types above it.

Nominal and ordinal attributes are collectively referred to as **categorical** or **qualitative** attributes. As the name suggests, qualitative attributes, such as employee ID, lack most of the properties of numbers. Even if they are represented by numbers, i.e., integers, they should be treated more like symbols. The remaining two types of attributes, interval and ratio, are collectively referred to as **quantitative** or **numeric** attributes. Quantitative attributes are represented by numbers and have most of the properties of numbers. Note that quantitative attributes can be integer-valued or continuous.

The types of attributes can also be described in terms of transformations that do not change the meaning of an attribute. Indeed, S. Smith Stevens, the

Table 2.3. Transformations that define attribute levels.

Attribute Type		Transformation	Comment
Categorical (Qualitative)	Nominal	Any one-to-one mapping, e.g., a permutation of values	If all employee ID numbers are reassigned, it will not make any difference.
	Ordinal	An order-preserving change of values, i.e., $new_value = f(old_value)$, where f is a monotonic function.	An attribute encompassing the notion of good, better, best can be represented equally well by the values $\{1, 2, 3\}$ or by $\{0.5, 1, 10\}$.
Numeric (Quantitative)	Interval	$new_value = a \times old_value + b$, a and b constants.	The Fahrenheit and Celsius temperature scales differ in the location of their zero value and the size of a degree (unit).
	Ratio	$new_value = a \times old_value$	Length can be measured in meters or feet.

psychologist who originally defined the types of attributes shown in Table 2.2, defined them in terms of these **permissible transformations**. For example, the meaning of a length attribute is unchanged if it is measured in meters instead of feet.

The statistical operations that make sense for a particular type of attribute are those that will yield the same results when the attribute is transformed by using a transformation that preserves the attribute's meaning. To illustrate, the average length of a set of objects is different when measured in meters rather than in feet, but both averages represent the same length. Table 2.3 shows the meaning-preserving transformations for the four attribute types of Table 2.2.

Example 2.5 (Temperature Scales)**.** Temperature provides a good illustration of some of the concepts that have been described. First, temperature can be either an interval or a ratio attribute, depending on its measurement scale. When measured on the Kelvin scale, a temperature of 2° is, in a physically meaningful way, twice that of a temperature of 1°. This is not true when temperature is measured on either the Celsius or Fahrenheit scales, because, physically, a temperature of 1° Fahrenheit (Celsius) is not much different than a temperature of 2° Fahrenheit (Celsius). The problem is that the zero points of the Fahrenheit and Celsius scales are, in a physical sense, arbitrary, and

therefore, the ratio of two Celsius or Fahrenheit temperatures is not physically meaningful. ■

Describing Attributes by the Number of Values

An independent way of distinguishing between attributes is by the number of values they can take.

Discrete A discrete attribute has a finite or countably infinite set of values. Such attributes can be categorical, such as zip codes or ID numbers, or numeric, such as counts. Discrete attributes are often represented using integer variables. **Binary attributes** are a special case of discrete attributes and assume only two values, e.g., true/false, yes/no, male/female, or 0/1. Binary attributes are often represented as Boolean variables, or as integer variables that only take the values 0 or 1.

Continuous A continuous attribute is one whose values are real numbers. Examples include attributes such as temperature, height, or weight. Continuous attributes are typically represented as floating-point variables. Practically, real values can be measured and represented only with limited precision.

In theory, any of the measurement scale types—nominal, ordinal, interval, and ratio—could be combined with any of the types based on the number of attribute values—binary, discrete, and continuous. However, some combinations occur only infrequently or do not make much sense. For instance, it is difficult to think of a realistic data set that contains a continuous binary attribute. Typically, nominal and ordinal attributes are binary or discrete, while interval and ratio attributes are continuous. However, **count attributes**, which are discrete, are also ratio attributes.

Asymmetric Attributes

For asymmetric attributes, only presence—a non-zero attribute value—is regarded as important. Consider a data set in which each object is a student and each attribute records whether a student took a particular course at a university. For a specific student, an attribute has a value of 1 if the student took the course associated with that attribute and a value of 0 otherwise. Because students take only a small fraction of all available courses, most of the values in such a data set would be 0. Therefore, it is more meaningful and more efficient to focus on the non-zero values. To illustrate, if students are compared

on the basis of the courses they don't take, then most students would seem very similar, at least if the number of courses is large. Binary attributes where only non-zero values are important are called **asymmetric binary attributes**. This type of attribute is particularly important for association analysis, which is discussed in Chapter 4. It is also possible to have discrete or continuous asymmetric features. For instance, if the number of credits associated with each course is recorded, then the resulting data set will consist of **asymmetric discrete** or **continuous attributes**.

General Comments on Levels of Measurement

As described in the rest of this chapter, there are many diverse types of data. The previous discussion of measurement scales, while useful, is not complete and has some limitations. We provide the following comments and guidance.

- **Distinctness, order, and meaningful intervals and ratios are only four properties of data—many others are possible**. For instance, some data is inherently cyclical, e.g., position on the surface of the Earth or time. As another example, consider set valued attributes, where each attribute value is a set of elements, e.g., the set of movies seen in the last year. Define one set of elements (movies) to be greater (larger) than a second set if the second set is a subset of the first. However, such a relationship defines only a partial order that does not match any of the attribute types just defined.

- **The numbers or symbols used to capture attribute values may not capture all the properties of the attributes or may suggest properties that are not there**. An illustration of this for integers was presented in Example 2.3, i.e., averages of IDs and out of range ages.

- **Data is often transformed for the purpose of analysis**—see Section 2.3.7. This often changes the distribution of the observed variable to a distribution that is easier to analyze, e.g., a Gaussian (normal) distribution. Often, such transformations only preserve the order of the original values, and other properties are lost. Nonetheless, if the desired outcome is a statistical test of differences or a predictive model, such a transformation is justified.

- **The final evaluation of any data analysis, including operations on attributes, is whether the results make sense from a domain point of view**.

In summary, it can be challenging to determine which operations can be performed on a particular attribute or a collection of attributes without compromising the integrity of the analysis. Fortunately, established practice often serves as a reliable guide. Occasionally, however, standard practices are erroneous or have limitations.

2.1.2 Types of Data Sets

There are many types of data sets, and as the field of data mining develops and matures, a greater variety of data sets become available for analysis. In this section, we describe some of the most common types. For convenience, we have grouped the types of data sets into three groups: record data, graph-based data, and ordered data. These categories do not cover all possibilities and other groupings are certainly possible.

General Characteristics of Data Sets

Before providing details of specific kinds of data sets, we discuss three characteristics that apply to many data sets and have a significant impact on the data mining techniques that are used: dimensionality, distribution, and resolution.

Dimensionality The dimensionality of a data set is the number of attributes that the objects in the data set possess. Analyzing data with a small number of dimensions tends to be qualitatively different from analyzing moderate or high-dimensional data. Indeed, the difficulties associated with the analysis of high-dimensional data are sometimes referred to as the **curse of dimensionality**. Because of this, an important motivation in preprocessing the data is **dimensionality reduction**. These issues are discussed in more depth later in this chapter and in Appendix B.

Distribution The distribution of a data set is the frequency of occurrence of various values or sets of values for the attributes comprising data objects. Equivalently, the distribution of a data set can be considered as a description of the concentration of objects in various regions of the data space. Statisticians have enumerated many types of distributions, e.g., Gaussian (normal), and described their properties. (See Appendix C.) Although statistical approaches for describing distributions can yield powerful analysis techniques, many data sets have distributions that are not well captured by standard statistical distributions.

As a result, many data mining algorithms do not assume a particular statistical distribution for the data they analyze. However, some general aspects of distributions often have a strong impact. For example, suppose a categorical attribute is used as a class variable, where one of the categories occurs 95% of the time, while the other categories together occur only 5% of the time. This **skewness** in the distribution can make classification difficult as discussed in Section 6.11. (Skewness has other impacts on data analysis that are not discussed here.)

A special case of skewed data is **sparsity**. For sparse binary, count or continuous data, most attributes of an object have values of 0. In many cases, fewer than 1% of the values are non-zero. In practical terms, sparsity is an advantage because usually only the non-zero values need to be stored and manipulated. This results in significant savings with respect to computation time and storage. Indeed, some data mining algorithms, such as the association rule mining algorithms described in Chapter 4, work well only for sparse data. Finally, note that often the attributes in sparse data sets are asymmetric attributes.

Resolution It is frequently possible to obtain data at different levels of resolution, and often the properties of the data are different at different resolutions. For instance, the surface of the Earth seems very uneven at a resolution of a few meters, but is relatively smooth at a resolution of tens of kilometers. The patterns in the data also depend on the level of resolution. If the resolution is too fine, a pattern may not be visible or may be buried in noise; if the resolution is too coarse, the pattern can disappear. For example, variations in atmospheric pressure on a scale of hours reflect the movement of storms and other weather systems. On a scale of months, such phenomena are not detectable.

Record Data

Much data mining work assumes that the data set is a collection of records (data objects), each of which consists of a fixed set of data fields (attributes). See Figure 2.2(a). For the most basic form of record data, there is no explicit relationship among records or data fields, and every record (object) has the same set of attributes. Record data is usually stored either in **flat** files or in relational databases. Relational databases are certainly more than a collection of records, but data mining often does not use any of the additional information available in a relational database. Rather, the database serves as

a convenient place to find records. Different types of record data are described below and are illustrated in Figure 2.2.

Tid	Refund	Marital Status	Taxable Income	Defaulted Borrower
1	Yes	Single	125K	**No**
2	No	Married	100K	**No**
3	No	Single	70K	**No**
4	Yes	Married	120K	**No**
5	No	Divorced	95K	**Yes**
6	No	Married	60K	**No**
7	Yes	Divorced	220K	**No**
8	No	Single	85K	**Yes**
9	No	Married	75K	**No**
10	No	Single	90K	**Yes**

(a) Record data.

TID	*ITEMS*
1	Bread, Soda, Milk
2	Beer, Bread
3	Beer, Soda, Diapers, Milk
4	Beer, Bread, Diapers, Milk
5	Soda, Diapers, Milk

(b) Transaction data.

Projection of x Load	Projection of y Load	Distance	Load	Thickness
10.23	5.27	15.22	27	1.2
12.65	6.25	16.22	22	1.1
13.54	7.23	17.34	23	1.2
14.27	8.43	18.45	25	0.9

(c) Data matrix.

	team	coach	play	ball	score	game	win	lost	timeout	season
Document 1	3	0	5	0	2	6	0	2	0	2
Document 2	0	7	0	2	1	0	0	3	0	0
Document 3	0	1	0	0	1	2	2	0	3	0

(d) Document-term matrix.

Figure 2.2. Different variations of record data.

Transaction or Market Basket Data Transaction data is a special type of record data, where each record (transaction) involves a set of items. Consider a grocery store. The set of products purchased by a customer during one shopping trip constitutes a transaction, while the individual products that were purchased are the items. This type of data is called **market basket data** because the items in each record are the products in a person's "market basket." Transaction data is a collection of sets of items, but it can be viewed as a set of records whose fields are asymmetric attributes. Most often, the attributes are binary, indicating whether an item was purchased, but more

generally, the attributes can be discrete or continuous, such as the number of items purchased or the amount spent on those items. Figure 2.2(b) shows a sample transaction data set. Each row represents the purchases of a particular customer at a particular time.

The Data Matrix If all the data objects in a collection of data have the same fixed set of numeric attributes, then the data objects can be thought of as points (vectors) in a multidimensional space, where each dimension represents a distinct attribute describing the object. A set of such data objects can be interpreted as an m by n matrix, where there are m rows, one for each object, and n columns, one for each attribute. (A representation that has data objects as columns and attributes as rows is also fine.) This matrix is called a **data matrix** or a **pattern matrix**. A data matrix is a variation of record data, but because it consists of numeric attributes, standard matrix operation can be applied to transform and manipulate the data. Therefore, the data matrix is the standard data format for most statistical data. Figure 2.2(c) shows a sample data matrix.

The Sparse Data Matrix A sparse data matrix is a special case of a data matrix where the attributes are of the same type and are asymmetric; i.e., only non-zero values are important. Transaction data is an example of a sparse data matrix that has only 0–1 entries. Another common example is document data. In particular, if the order of the terms (words) in a document is ignored—the "bag of words" approach—then a document can be represented as a term vector, where each term is a component (attribute) of the vector and the value of each component is the number of times the corresponding term occurs in the document. This representation of a collection of documents is often called a **document-term matrix**. Figure 2.2(d) shows a sample document-term matrix. The documents are the rows of this matrix, while the terms are the columns. In practice, only the non-zero entries of sparse data matrices are stored.

Graph-Based Data

A graph can sometimes be a convenient and powerful representation for data. We consider two specific cases: (1) the graph captures relationships among data objects and (2) the data objects themselves are represented as graphs.

Data with Relationships among Objects The relationships among objects frequently convey important information. In such cases, the data is often

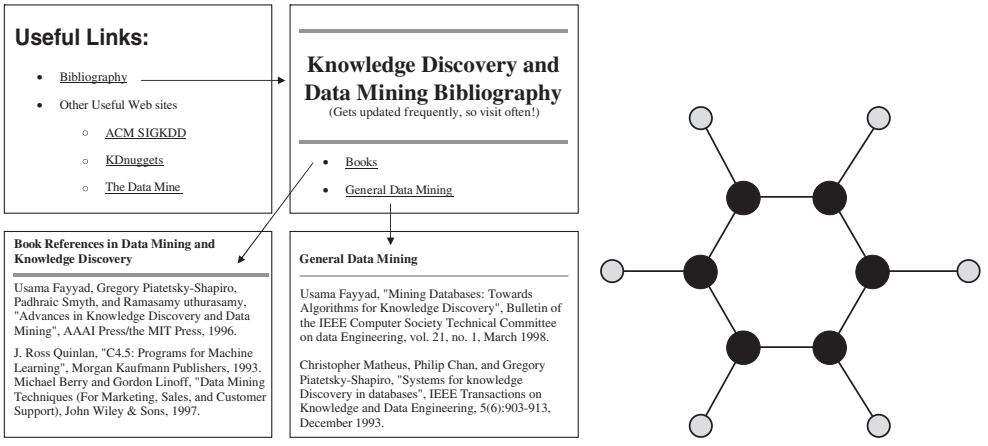
represented as a graph. In particular, the data objects are mapped to nodes of the graph, while the relationships among objects are captured by the links between objects and link properties, such as direction and weight. Consider web pages on the World Wide Web, which contain both text and links to other pages. In order to process search queries, web search engines collect and process web pages to extract their contents. It is well-known, however, that the links to and from each page provide a great deal of information about the relevance of a web page to a query, and thus, must also be taken into consideration. Figure 2.3(a) shows a set of linked web pages. Another important example of such graph data are the social networks, where data objects are people and the relationships among them are their interactions via social media.

Data with Objects That Are Graphs If objects have structure, that is, the objects contain subobjects that have relationships, then such objects are frequently represented as graphs. For example, the structure of chemical compounds can be represented by a graph, where the nodes are atoms and the links between nodes are chemical bonds. Figure 2.3(b) shows a ball-and-stick diagram of the chemical compound benzene, which contains atoms of carbon (black) and hydrogen (gray). A graph representation makes it possible to determine which substructures occur frequently in a set of compounds and to ascertain whether the presence of any of these substructures is associated with the presence or absence of certain chemical properties, such as melting point or heat of formation. Frequent graph mining, which is a branch of data mining that analyzes such data, is considered in Section 7.5.

Ordered Data

For some types of data, the attributes have relationships that involve order in time or space. Different types of ordered data are described next and are shown in Figure 2.4.

Sequential Transaction Data Sequential transaction data can be thought of as an extension of transaction data, where each transaction has a time associated with it. Consider a retail transaction data set that also stores the time at which the transaction took place. This time information makes it possible to find patterns such as "candy sales peak before Halloween." A time can also be associated with each attribute. For example, each record could be the purchase history of a customer, with a listing of items purchased at different times. Using this information, it is possible to find patterns such as



(a) Linked web pages. (b) Benzene molecule.

Figure 2.3. Different variations of graph data.

"people who buy DVD players tend to buy DVDs in the period immediately following the purchase."

Figure 2.4(a) shows an example of sequential transaction data. There are five different times—*t1*, *t2*, *t3*, *t4*, and *t5*; three different customers—C1, C2, and C3; and five different items—A, B, C, D, and E. In the top table, each row corresponds to the items purchased at a particular time by each customer. For instance, at time *t3*, customer C2 purchased items A and D. In the bottom table, the same information is displayed, but each row corresponds to a particular customer. Each row contains information about each transaction involving the customer, where a transaction is considered to be a set of items and the time at which those items were purchased. For example, customer C3 bought items A and C at time *t2*.

Time Series Data Time series data is a special type of ordered data where each record is a **time series**, i.e., a series of measurements taken over time. For example, a financial data set might contain objects that are time series of the daily prices of various stocks. As another example, consider Figure 2.4(c), which shows a time series of the average monthly temperature for Minneapolis during the years 1982 to 1994. When working with temporal data, such as time series, it is important to consider **temporal autocorrelation**; i.e., if

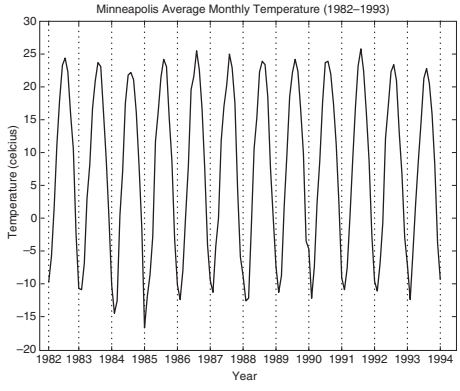
Time	Customer	Items Purchased
t1	C1	A, B
t2	C3	A, C
t2	C1	C, D
t3	C2	A, D
t4	C2	E
t5	C1	A, E

Customer	Time and Items Purchased
C1	(t1: A,B) (t2:C,D) (t5:A,E)
C2	(t3: A, D) (t4: E)
C3	(t2: A, C)

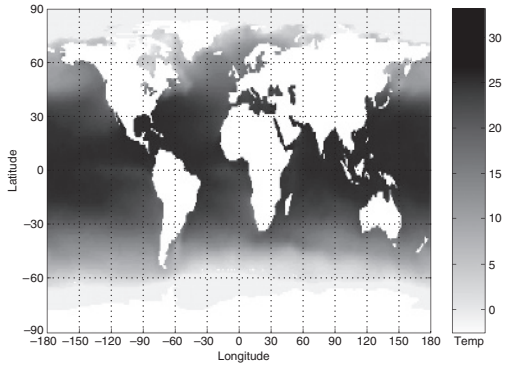
(a) Sequential transaction data.

```
GGTTCCGCCTTCAGCCCCGCGCC
CGCAGGGCCCGCCCCGCGCCGTC
GAGAAGGGCCCGCCTGGCGGGCG
GGGGGAGGCGGGGCCGCCCGAGC
CCAACCGAGTCCGACCAGGTGCC
CCCTCTGCTCGGCCTAGACCTGA
GCTCATTAGGCGGCAGCGGACAG
GCCAAGTAGAACACGCGAAGCGC
TGGGCTGCCTGCTGCGACCAGGG
```

(b) Genomic sequence data.



(c) Temperature time series.



(d) Spatial temperature data.

Figure 2.4. Different variations of ordered data.

two measurements are close in time, then the values of those measurements are often very similar.

Sequence Data Sequence data consists of a data set that is a sequence of individual entities, such as a sequence of words or letters. It is quite similar to sequential data, except that there are no time stamps; instead, there are positions in an ordered sequence. For example, the genetic information of plants and animals can be represented in the form of sequences of nucleotides

that are known as genes. Many of the problems associated with genetic sequence data involve predicting similarities in the structure and function of genes from similarities in nucleotide sequences. Figure 2.4(b) shows a section of the human genetic code expressed using the four nucleotides from which all DNA is constructed: A, T, G, and C.

Spatial and Spatio-Temporal Data Some objects have spatial attributes, such as positions or areas, in addition to other types of attributes. An example of spatial data is weather data (precipitation, temperature, pressure) that is collected for a variety of geographical locations. Often such measurements are collected over time, and thus, the data consists of time series at various locations. In that case, we refer to the data as spatio-temporal data. Although analysis can be conducted separately for each specific time or location, a more complete analysis of spatio-temporal data requires consideration of both the spatial and temporal aspects of the data.

An important aspect of spatial data is **spatial autocorrelation**; i.e., objects that are physically close tend to be similar in other ways as well. Thus, two points on the Earth that are close to each other usually have similar values for temperature and rainfall. Note that spatial autocorrelation is analogous to temporal autocorrelation.

Important examples of spatial and spatio-temporal data are the science and engineering data sets that are the result of measurements or model output taken at regularly or irregularly distributed points on a two- or three-dimensional grid or mesh. For instance, Earth science data sets record the temperature or pressure measured at points (grid cells) on latitude–longitude spherical grids of various resolutions, e.g., 1° by 1°. See Figure 2.4(d). As another example, in the simulation of the flow of a gas, the speed and direction of flow at various instants in time can be recorded for each grid point in the simulation. A different type of spatio-temporal data arises from tracking the trajectories of objects, e.g., vehicles, in time and space.

Handling Non-Record Data

Most data mining algorithms are designed for record data or its variations, such as transaction data and data matrices. Record-oriented techniques can be applied to non-record data by extracting features from data objects and using these features to create a record corresponding to each object. Consider the chemical structure data that was described earlier. Given a set of common substructures, each compound can be represented as a record with binary attributes that indicate whether a compound contains a specific substructure.

Such a representation is actually a transaction data set, where the transactions are the compounds and the items are the substructures.

In some cases, it is easy to represent the data in a record format, but this type of representation does not capture all the information in the data. Consider spatio-temporal data consisting of a time series from each point on a spatial grid. This data is often stored in a data matrix, where each row represents a location and each column represents a particular point in time. However, such a representation does not explicitly capture the time relationships that are present among attributes and the spatial relationships that exist among objects. This does not mean that such a representation is inappropriate, but rather that these relationships must be taken into consideration during the analysis. For example, it would not be a good idea to use a data mining technique that ignores the temporal autocorrelation of the attributes or the spatial autocorrelation of the data objects, i.e., the locations on the spatial grid.

2.2 Data Quality

Data mining algorithms are often applied to data that was collected for another purpose, or for future, but unspecified applications. For that reason, data mining cannot usually take advantage of the significant benefits of "addressing quality issues at the source." In contrast, much of statistics deals with the design of experiments or surveys that achieve a prespecified level of data quality. Because preventing data quality problems is typically not an option, data mining focuses on (1) the detection and correction of data quality problems and (2) the use of algorithms that can tolerate poor data quality. The first step, detection and correction, is often called **data cleaning**.

The following sections discuss specific aspects of data quality. The focus is on measurement and data collection issues, although some application-related issues are also discussed.

2.2.1 Measurement and Data Collection Issues

It is unrealistic to expect that data will be perfect. There may be problems due to human error, limitations of measuring devices, or flaws in the data collection process. Values or even entire data objects can be missing. In other cases, there can be spurious or duplicate objects; i.e., multiple data objects that all correspond to a single "real" object. For example, there might be two different records for a person who has recently lived at two different addresses.

Even if all the data is present and "looks fine," there may be inconsistencies—a person has a height of 2 meters, but weighs only 2 kilograms.

In the next few sections, we focus on aspects of data quality that are related to data measurement and collection. We begin with a definition of measurement and data collection errors and then consider a variety of problems that involve measurement error: noise, artifacts, bias, precision, and accuracy. We conclude by discussing data quality issues that involve both measurement and data collection problems: outliers, missing and inconsistent values, and duplicate data.

Measurement and Data Collection Errors

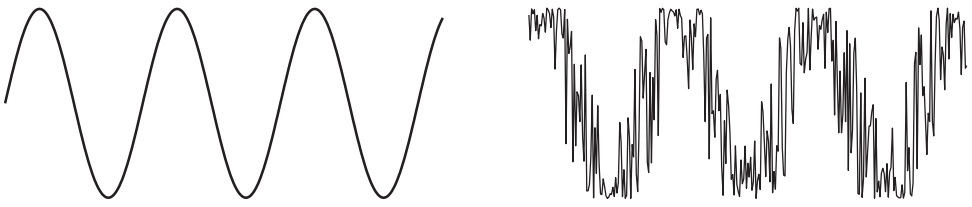
The term **measurement error** refers to any problem resulting from the measurement process. A common problem is that the value recorded differs from the true value to some extent. For continuous attributes, the numerical difference of the measured and true value is called the **error**. The term **data collection error** refers to errors such as omitting data objects or attribute values, or inappropriately including a data object. For example, a study of animals of a certain species might include animals of a related species that are similar in appearance to the species of interest. Both measurement errors and data collection errors can be either systematic or random.

We will only consider general types of errors. Within particular domains, certain types of data errors are commonplace, and well-developed techniques often exist for detecting and/or correcting these errors. For example, keyboard errors are common when data is entered manually, and as a result, many data entry programs have techniques for detecting and, with human intervention, correcting such errors.

Noise and Artifacts

Noise is the random component of a measurement error. It typically involves the distortion of a value or the addition of spurious objects. Figure 2.5 shows a time series before and after it has been disrupted by random noise. If a bit more noise were added to the time series, its shape would be lost. Figure 2.6 shows a set of data points before and after some noise points (indicated by '+'s) have been added. Notice that some of the noise points are intermixed with the non-noise points.

The term noise is often used in connection with data that has a spatial or temporal component. In such cases, techniques from signal or image processing



(a) Time series. (b) Time series with noise.

Figure 2.5. Noise in a time series context.



(a) Three groups of points. (b) With noise points (+) added.

Figure 2.6. Noise in a spatial context.

can frequently be used to reduce noise and thus, help to discover patterns (signals) that might be "lost in the noise." Nonetheless, the elimination of noise is frequently difficult, and much work in data mining focuses on devising **robust algorithms** that produce acceptable results even when noise is present.

Data errors can be the result of a more deterministic phenomenon, such as a streak in the same place on a set of photographs. Such deterministic distortions of the data are often referred to as **artifacts**.

Precision, Bias, and Accuracy

In statistics and experimental science, the quality of the measurement process and the resulting data are measured by precision and bias. We provide the

standard definitions, followed by a brief discussion. For the following definitions, we assume that we make repeated measurements of the same underlying quantity.

Definition 2.3 (Precision)**.** The closeness of repeated measurements (of the same quantity) to one another.

Definition 2.4 (Bias)**.** A systematic variation of measurements from the quantity being measured.

Precision is often measured by the standard deviation of a set of values, while bias is measured by taking the difference between the mean of the set of values and the known value of the quantity being measured. Bias can be determined only for objects whose measured quantity is known by means external to the current situation. Suppose that we have a standard laboratory weight with a mass of 1g and want to assess the precision and bias of our new laboratory scale. We weigh the mass five times, and obtain the following five values: $\{1.015, 0.990, 1.013, 1.001, 0.986\}$. The mean of these values is 1.001, and hence, the bias is 0.001. The precision, as measured by the standard deviation, is 0.013.

It is common to use the more general term, **accuracy**, to refer to the degree of measurement error in data.

Definition 2.5 (Accuracy)**.** The closeness of measurements to the true value of the quantity being measured.

Accuracy depends on precision and bias, but there is no specific formula for accuracy in terms of these two quantities.

One important aspect of accuracy is the use of **significant digits**. The goal is to use only as many digits to represent the result of a measurement or calculation as are justified by the precision of the data. For example, if the length of an object is measured with a meter stick whose smallest markings are millimeters, then we should record the length of data only to the nearest millimeter. The precision of such a measurement would be $\pm$ 0.5mm. We do not review the details of working with significant digits because most readers will have encountered them in previous courses and they are covered in considerable depth in science, engineering, and statistics textbooks.

Issues such as significant digits, precision, bias, and accuracy are sometimes overlooked, but they are important for data mining as well as statistics and science. Many times, data sets do not come with information about the precision of the data, and furthermore, the programs used for analysis return results without any such information. Nonetheless, without some understanding of the

accuracy of the data and the results, an analyst runs the risk of committing serious data analysis blunders.

Outliers

Outliers are either (1) data objects that, in some sense, have characteristics that are different from most of the other data objects in the data set, or (2) values of an attribute that are unusual with respect to the typical values for that attribute. Alternatively, they can be referred to as **anomalous** objects or values. There is considerable leeway in the definition of an outlier, and many different definitions have been proposed by the statistics and data mining communities. Furthermore, it is important to distinguish between the notions of noise and outliers. Unlike noise, outliers can be legitimate data objects or values that we are interested in detecting. For instance, in fraud and network intrusion detection, the goal is to find unusual objects or events from among a large number of normal ones. Chapter 9 discusses anomaly detection in more detail.

Missing Values

It is not unusual for an object to be missing one or more attribute values. In some cases, the information was not collected; e.g., some people decline to give their age or weight. In other cases, some attributes are not applicable to all objects; e.g., often, forms have conditional parts that are filled out only when a person answers a previous question in a certain way, but for simplicity, all fields are stored. Regardless, missing values should be taken into account during the data analysis.

There are several strategies (and variations on these strategies) for dealing with missing data, each of which is appropriate in certain circumstances. These strategies are listed next, along with an indication of their advantages and disadvantages.

Eliminate Data Objects or Attributes A simple and effective strategy is to eliminate objects with missing values. However, even a partially specified data object contains some information, and if many objects have missing values, then a reliable analysis can be difficult or impossible. Nonetheless, if a data set has only a few objects that have missing values, then it may be expedient to omit them. A related strategy is to eliminate attributes that have missing values. This should be done with caution, however, because the eliminated attributes may be the ones that are critical to the analysis.

Estimate Missing Values Sometimes missing data can be reliably estimated. For example, consider a time series that changes in a reasonably smooth fashion, but has a few, widely scattered missing values. In such cases, the missing values can be estimated (interpolated) by using the remaining values. As another example, consider a data set that has many similar data points. In this situation, the attribute values of the points closest to the point with the missing value are often used to estimate the missing value. If the attribute is continuous, then the average attribute value of the nearest neighbors is used; if the attribute is categorical, then the most commonly occurring attribute value can be taken. For a concrete illustration, consider precipitation measurements that are recorded by ground stations. For areas not containing a ground station, the precipitation can be estimated using values observed at nearby ground stations.

Ignore the Missing Value during Analysis Many data mining approaches can be modified to ignore missing values. For example, suppose that objects are being clustered and the similarity between pairs of data objects needs to be calculated. If one or both objects of a pair have missing values for some attributes, then the similarity can be calculated by using only the attributes that do not have missing values. It is true that the similarity will only be approximate, but unless the total number of attributes is small or the number of missing values is high, this degree of inaccuracy may not matter much. Likewise, many classification schemes can be modified to work with missing values.

Inconsistent Values

Data can contain inconsistent values. Consider an address field, where both a zip code and city are listed, but the specified zip code area is not contained in that city. It is possible that the individual entering this information transposed two digits, or perhaps a digit was misread when the information was scanned from a handwritten form. Regardless of the cause of the inconsistent values, it is important to detect and, if possible, correct such problems.

Some types of inconsistences are easy to detect. For instance, a person's height should not be negative. In other cases, it can be necessary to consult an external source of information. For example, when an insurance company processes claims for reimbursement, it checks the names and addresses on the reimbursement forms against a database of its customers.

Once an inconsistency has been detected, it is sometimes possible to correct the data. A product code may have "check" digits, or it may be possible to

double-check a product code against a list of known product codes, and then correct the code if it is incorrect, but close to a known code. The correction of an inconsistency requires additional or redundant information.

Example 2.6 (Inconsistent Sea Surface Temperature)**.** This example illustrates an inconsistency in actual time series data that measures the sea surface temperature (SST) at various points on the ocean. SST data was originally collected using ocean-based measurements from ships or buoys, but more recently, satellites have been used to gather the data. To create a long-term data set, both sources of data must be used. However, because the data comes from different sources, the two parts of the data are subtly different. This discrepancy is visually displayed in Figure 2.7, which shows the correlation of SST values between pairs of years. If a pair of years has a positive correlation, then the location corresponding to the pair of years is colored white; otherwise it is colored black. (Seasonal variations were removed from the data since, otherwise, all the years would be highly correlated.) There is a distinct change in behavior where the data has been put together in 1983. Years within each of the two groups, 1958–1982 and 1983–1999, tend to have a positive correlation with one another, but a negative correlation with years in the other group. This does not mean that this data should not be used, only that the analyst should consider the potential impact of such discrepancies on the data mining analysis. ■

Duplicate Data

A data set can include data objects that are duplicates, or almost duplicates, of one another. Many people receive duplicate mailings because they appear in a database multiple times under slightly different names. To detect and eliminate such duplicates, two main issues must be addressed. First, if there are two objects that actually represent a single object, then one or more values of corresponding attributes are usually different, and these inconsistent values must be resolved. Second, care needs to be taken to avoid accidentally combining data objects that are similar, but not duplicates, such as two distinct people with identical names. The term **deduplication** is often used to refer to the process of dealing with these issues.

In some cases, two or more objects are identical with respect to the attributes measured by the database, but they still represent different objects. Here, the duplicates are legitimate, but can still cause problems for some algorithms if the possibility of identical objects is not specifically accounted for in their design. An example of this is given in Exercise 17 on page 128.

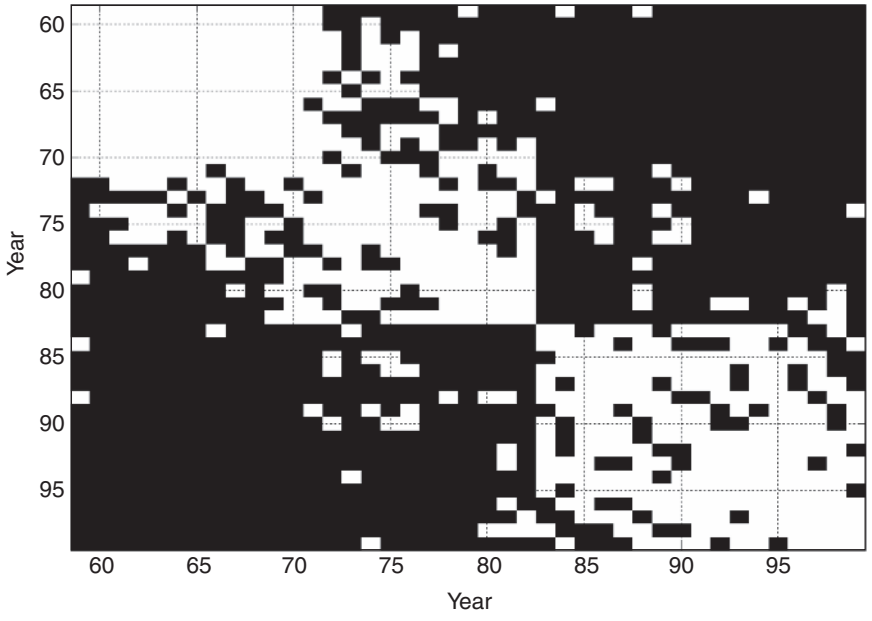


Figure 2.7. Correlation of SST data between pairs of years. White areas indicate positive correlation. Black areas indicate negative correlation.

2.2.2 Issues Related to Applications

Data quality issues can also be considered from an application viewpoint as expressed by the statement "data is of high quality if it is suitable for its intended use." This approach to data quality has proven quite useful, particularly in business and industry. A similar viewpoint is also present in statistics and the experimental sciences, with their emphasis on the careful design of experiments to collect the data relevant to a specific hypothesis. As with quality issues at the measurement and data collection level, many issues are specific to particular applications and fields. Again, we consider only a few of the general issues.

Timeliness Some data starts to age as soon as it has been collected. In particular, if the data provides a snapshot of some ongoing phenomenon or process, such as the purchasing behavior of customers or web browsing patterns, then this snapshot represents reality for only a limited time. If the data is out of date, then so are the models and patterns that are based on it.

Relevance The available data must contain the information necessary for the application. Consider the task of building a model that predicts the accident rate for drivers. If information about the age and gender of the driver is omitted, then it is likely that the model will have limited accuracy unless this information is indirectly available through other attributes.

Making sure that the objects in a data set are relevant is also challenging. A common problem is **sampling bias**, which occurs when a sample does not contain different types of objects in proportion to their actual occurrence in the population. For example, survey data describes only those who respond to the survey. (Other aspects of sampling are discussed further in Section 2.3.2.) Because the results of a data analysis can reflect only the data that is present, sampling bias will typically lead to erroneous results when applied to the broader population.

Knowledge about the Data Ideally, data sets are accompanied by documentation that describes different aspects of the data; the quality of this documentation can either aid or hinder the subsequent analysis. For example, if the documentation identifies several attributes as being strongly related, these attributes are likely to provide highly redundant information, and we usually decide to keep just one. (Consider sales tax and purchase price.) If the documentation is poor, however, and fails to tell us, for example, that the missing values for a particular field are indicated with a -9999, then our analysis of the data may be faulty. Other important characteristics are the precision of the data, the type of features (nominal, ordinal, interval, ratio), the scale of measurement (e.g., meters or feet for length), and the origin of the data.

2.3 Data Preprocessing

In this section, we consider which preprocessing steps should be applied to make the data more suitable for data mining. Data preprocessing is a broad area and consists of a number of different strategies and techniques that are interrelated in complex ways. We will present some of the most important ideas and approaches, and try to point out the interrelationships among them. Specifically, we will discuss the following topics:

- Aggregation
- Sampling
- Dimensionality reduction

- Feature subset selection
- Feature creation
- Discretization and binarization
- Variable transformation

Roughly speaking, these topics fall into two categories: selecting data objects and attributes for the analysis or for creating/changing the attributes. In both cases, the goal is to improve the data mining analysis with respect to time, cost, and quality. Details are provided in the following sections.

A quick note about terminology: In the following, we sometimes use synonyms for attribute, such as feature or variable, in order to follow common usage.

2.3.1 Aggregation

Sometimes "less is more," and this is the case with **aggregation**, the combining of two or more objects into a single object. Consider a data set consisting of transactions (data objects) recording the daily sales of products in various store locations (Minneapolis, Chicago, Paris, ...) for different days over the course of a year. See Table 2.4. One way to aggregate transactions for this data set is to replace all the transactions of a single store with a single storewide transaction. This reduces the hundreds or thousands of transactions that occur daily at a specific store to a single daily transaction, and the number of data objects per day is reduced to the number of stores.

An obvious issue is how an aggregate transaction is created; i.e., how the values of each attribute are combined across all the records corresponding to a particular location to create the aggregate transaction that represents the sales of a single store or date. Quantitative attributes, such as *price*, are typically aggregated by taking a sum or an average. A qualitative attribute, such as *item*, can either be omitted or summarized in terms of a higher level category, e.g., televisions versus electronics.

The data in Table 2.4 can also be viewed as a multidimensional array, where each attribute is a dimension. From this viewpoint, aggregation is the process of eliminating attributes, such as the type of item, or reducing the number of values for a particular attribute; e.g., reducing the possible values for *date* from 365 days to 12 months. This type of aggregation is commonly used in Online Analytical Processing (OLAP). References to OLAP are given in the Bibliographic Notes.

There are several motivations for aggregation. First, the smaller data sets resulting from data reduction require less memory and processing time,

Table 2.4. Data set containing information about customer purchases.

Transaction ID	Item	Store Location	Date	Price	...
⋮	⋮	⋮	⋮	⋮	
101123	Watch	Chicago	09/06/04	$25.99	...
101123	Battery	Chicago	09/06/04	$5.99	...
101124	Shoes	Minneapolis	09/06/04	$75.00	...
⋮	⋮	⋮	⋮	⋮	

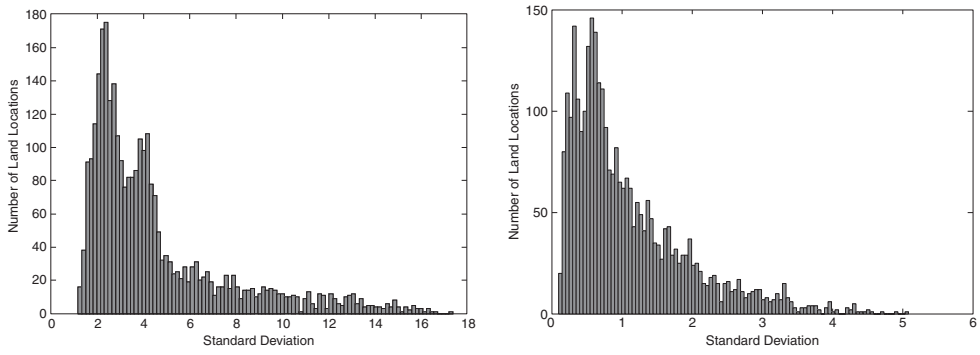
and hence, aggregation often enables the use of more expensive data mining algorithms. Second, aggregation can act as a change of scope or scale by providing a high-level view of the data instead of a low-level view. In the previous example, aggregating over store locations and months gives us a monthly, per store view of the data instead of a daily, per item view. Finally, the behavior of groups of objects or attributes is often more stable than that of individual objects or attributes. This statement reflects the statistical fact that aggregate quantities, such as averages or totals, have less variability than the individual values being aggregated. For totals, the actual amount of variation is larger than that of individual objects (on average), but the percentage of the variation is smaller, while for means, the actual amount of variation is less than that of individual objects (on average). A disadvantage of aggregation is the potential loss of interesting details. In the store example, aggregating over months loses information about which day of the week has the highest sales.

Example 2.7 (Australian Precipitation)**.** This example is based on precipitation in Australia from the period 1982–1993. Figure 2.8(a) shows a histogram for the standard deviation of average monthly precipitation for 3,030 0.5° by 0.5° grid cells in Australia, while Figure 2.8(b) shows a histogram for the standard deviation of the average yearly precipitation for the same locations. The average yearly precipitation has less variability than the average monthly precipitation. All precipitation measurements (and their standard deviations) are in centimeters.

■

2.3.2 Sampling

Sampling is a commonly used approach for selecting a subset of the data objects to be analyzed. In statistics, it has long been used for both the preliminary investigation of the data and the final data analysis. Sampling can also be very useful in data mining. However, the motivations for sampling



(a) Histogram of standard deviation of average monthly precipitation

(b) Histogram of standard deviation of average yearly precipitation

Figure 2.8. Histograms of standard deviation for monthly and yearly precipitation in Australia for the period 1982–1993.

in statistics and data mining are often different. Statisticians use sampling because obtaining the entire set of data of interest is too expensive or time consuming, while data miners usually sample because it is too computationally expensive in terms of the memory or time required to process all the data. In some cases, using a sampling algorithm can reduce the data size to the point where a better, but more computationally expensive algorithm can be used.

The key principle for effective sampling is the following: Using a sample will work almost as well as using the entire data set if the sample is representative. In turn, **a sample is representative** if it has approximately the same property (of interest) as the original set of data. If the mean (average) of the data objects is the property of interest, then a sample is representative if it has a mean that is close to that of the original data. Because sampling is a statistical process, the representativeness of any particular sample will vary, and the best that we can do is choose a sampling scheme that guarantees a high probability of getting a representative sample. As discussed next, this involves choosing the appropriate sample size and sampling technique.

Sampling Approaches

There are many sampling techniques, but only a few of the most basic ones and their variations will be covered here. The simplest type of sampling is **simple random sampling**. For this type of sampling, there is an equal

probability of selecting any particular object. There are two variations on random sampling (and other sampling techniques as well): (1) **sampling without replacement**—as each object is selected, it is removed from the set of all objects that together constitute the **population**, and (2) **sampling with replacement**—objects are not removed from the population as they are selected for the sample. In sampling with replacement, the same object can be picked more than once. The samples produced by the two methods are not much different when samples are relatively small compared to the data set size, but sampling with replacement is simpler to analyze because the probability of selecting any object remains constant during the sampling process.

When the population consists of different types of objects, with widely different numbers of objects, simple random sampling can fail to adequately represent those types of objects that are less frequent. This can cause problems when the analysis requires proper representation of all object types. For example, when building classification models for rare classes, it is critical that the rare classes be adequately represented in the sample. Hence, a sampling scheme that can accommodate differing frequencies for the object types of interest is needed. **Stratified sampling**, which starts with prespecified groups of objects, is such an approach. In the simplest version, equal numbers of objects are drawn from each group even though the groups are of different sizes. In another variation, the number of objects drawn from each group is proportional to the size of that group.

Example 2.8 (Sampling and Loss of Information)**.** Once a sampling technique has been selected, it is still necessary to choose the sample size. Larger sample sizes increase the probability that a sample will be representative, but they also eliminate much of the advantage of sampling. Conversely, with smaller sample sizes, patterns can be missed or erroneous patterns can be detected. Figure 2.9(a) shows a data set that contains 8000 two-dimensional points, while Figures 2.9(b) and 2.9(c) show samples from this data set of size 2000 and 500, respectively. Although most of the structure of this data set is present in the sample of 2000 points, much of the structure is missing in the sample of 500 points. ■

Example 2.9 (Determining the Proper Sample Size)**.** To illustrate that determining the proper sample size requires a methodical approach, consider the following task.

> Given a set of data consisting of a small number of almost equal-sized groups, find at least one representative point for each of the groups. Assume that the objects in each group are highly similar



(a) 8000 points (b) 2000 points (c) 500 points

Figure 2.9. Example of the loss of structure with sampling.

to each other, but not very similar to objects in different groups. Figure 2.10(a) shows an idealized set of clusters (groups) from which these points might be drawn.

This problem can be efficiently solved using sampling. One approach is to take a small sample of data points, compute the pairwise similarities between points, and then form groups of points that are highly similar. The desired set of representative points is then obtained by taking one point from each of these groups. To follow this approach, however, we need to determine a sample size that would guarantee, with a high probability, the desired outcome; that is, that at least one point will be obtained from each cluster. Figure 2.10(b) shows the probability of getting one object from each of the 10 groups as the sample size runs from 10 to 60. Interestingly, with a sample size of 20, there is little chance (20%) of getting a sample that includes all 10 clusters. Even with a sample size of 30, there is still a moderate chance (almost 40%) of getting a sample that doesn't contain objects from all 10 clusters. This issue is further explored in the context of clustering by Exercise 4 on page 126.

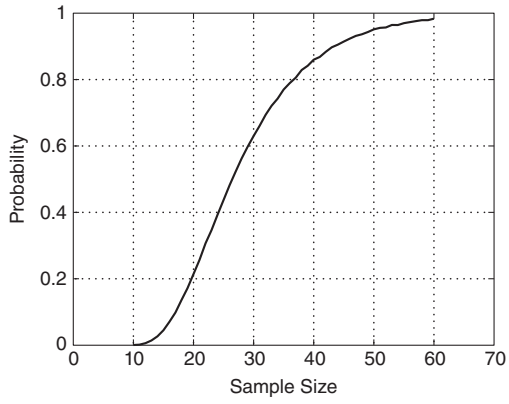
■

Progressive Sampling

The proper sample size can be difficult to determine, so **adaptive** or **progressive sampling** schemes are sometimes used. These approaches start with a small sample, and then increase the sample size until a sample of sufficient size has been obtained. While this technique eliminates the need to determine



(a) Ten groups of points.



(b) Probability a sample contains points from each of 10 groups.

Figure 2.10. Finding representative points from 10 groups.

the correct sample size initially, it requires that there be a way to evaluate the sample to judge if it is large enough.

Suppose, for instance, that progressive sampling is used to learn a predictive model. Although the accuracy of predictive models increases as the sample size increases, at some point the increase in accuracy levels off. We want to stop increasing the sample size at this leveling-off point. By keeping track of the change in accuracy of the model as we take progressively larger samples, and by taking other samples close to the size of the current one, we can get an estimate of how close we are to this leveling-off point, and thus, stop sampling.

2.3.3 Dimensionality Reduction

Data sets can have a large number of features. Consider a set of documents, where each document is represented by a vector whose components are the frequencies with which each word occurs in the document. In such cases, there are typically thousands or tens of thousands of attributes (components), one for each word in the vocabulary. As another example, consider a set of time series consisting of the daily closing price of various stocks over a period of 30 years. In this case, the attributes, which are the prices on specific days, again number in the thousands.

There are a variety of benefits to dimensionality reduction. A key benefit is that many data mining algorithms work better if the dimensionality—the number of attributes in the data—is lower. This is partly because dimensionality reduction can eliminate irrelevant features and reduce noise and partly because of the curse of dimensionality, which is explained below. Another benefit is that a reduction of dimensionality can lead to a more understandable model because the model usually involves fewer attributes. Also, dimensionality reduction may allow the data to be more easily visualized. Even if dimensionality reduction doesn't reduce the data to two or three dimensions, data is often visualized by looking at pairs or triplets of attributes, and the number of such combinations is greatly reduced. Finally, the amount of time and memory required by the data mining algorithm is reduced with a reduction in dimensionality.

The term dimensionality reduction is often reserved for those techniques that reduce the dimensionality of a data set by creating new attributes that are a combination of the old attributes. The reduction of dimensionality by selecting attributes that are a subset of the old is known as feature subset selection or feature selection. It will be discussed in Section 2.3.4.

In the remainder of this section, we briefly introduce two important topics: the curse of dimensionality and dimensionality reduction techniques based on linear algebra approaches such as principal components analysis (PCA). More details on dimensionality reduction can be found in Appendix B.

The Curse of Dimensionality

The curse of dimensionality refers to the phenomenon that many types of data analysis become significantly harder as the dimensionality of the data increases. Specifically, as dimensionality increases, the data becomes increasingly sparse in the space that it occupies. Thus, the data objects we observe are quite possibly not a representative sample of all possible objects. For classification, this can mean that there are not enough data objects to allow the creation of a model that reliably assigns a class to all possible objects. For clustering, the differences in density and in the distances between points, which are critical for clustering, become less meaningful. (This is discussed further in Sections 8.1.2, 8.4.6, and 8.4.8.) As a result, many clustering and classification algorithms (and other data analysis algorithms) have trouble with high-dimensional data leading to reduced classification accuracy and poor quality clusters.

Linear Algebra Techniques for Dimensionality Reduction

Some of the most common approaches for dimensionality reduction, particularly for continuous data, use techniques from linear algebra to project the data from a high-dimensional space into a lower-dimensional space. **Principal Components Analysis (PCA)** is a linear algebra technique for continuous attributes that finds new attributes (principal components) that (1) are linear combinations of the original attributes, (2) are **orthogonal** (perpendicular) to each other, and (3) capture the maximum amount of variation in the data. For example, the first two principal components capture as much of the variation in the data as is possible with two orthogonal attributes that are linear combinations of the original attributes. **Singular Value Decomposition (SVD)** is a linear algebra technique that is related to PCA and is also commonly used for dimensionality reduction. For additional details, see Appendices A and B.

2.3.4 Feature Subset Selection

Another way to reduce the dimensionality is to use only a subset of the features. While it might seem that such an approach would lose information, this is not the case if redundant and irrelevant features are present. **Redundant features** duplicate much or all of the information contained in one or more other attributes. For example, the purchase price of a product and the amount of sales tax paid contain much of the same information. **Irrelevant features** contain almost no useful information for the data mining task at hand. For instance, students' ID numbers are irrelevant to the task of predicting students' grade point averages. Redundant and irrelevant features can reduce classification accuracy and the quality of the clusters that are found.

While some irrelevant and redundant attributes can be eliminated immediately by using common sense or domain knowledge, selecting the best subset of features frequently requires a systematic approach. The ideal approach to feature selection is to try all possible subsets of features as input to the data mining algorithm of interest, and then take the subset that produces the best results. This method has the advantage of reflecting the objective and bias of the data mining algorithm that will eventually be used. Unfortunately, since the number of subsets involving n attributes is 2^n, such an approach is impractical in most situations and alternative strategies are needed. There are three standard approaches to feature selection: embedded, filter, and wrapper.

Embedded approaches Feature selection occurs naturally as part of the data mining algorithm. Specifically, during the operation of the data mining algorithm, the algorithm itself decides which attributes to use and which to ignore. Algorithms for building decision tree classifiers, which are discussed in Chapter 3, often operate in this manner.

Filter approaches Features are selected before the data mining algorithm is run, using some approach that is independent of the data mining task. For example, we might select sets of attributes whose pairwise correlation is as low as possible so that the attributes are non-redundant.

Wrapper approaches These methods use the target data mining algorithm as a black box to find the best subset of attributes, in a way similar to that of the ideal algorithm described above, but typically without enumerating all possible subsets.

Because the embedded approaches are algorithm-specific, only the filter and wrapper approaches will be discussed further here.

An Architecture for Feature Subset Selection

It is possible to encompass both the filter and wrapper approaches within a common architecture. The feature selection process is viewed as consisting of four parts: a measure for evaluating a subset, a search strategy that controls the generation of a new subset of features, a stopping criterion, and a validation procedure. Filter methods and wrapper methods differ only in the way in which they evaluate a subset of features. For a wrapper method, subset evaluation uses the target data mining algorithm, while for a filter approach, the evaluation technique is distinct from the target data mining algorithm. The following discussion provides some details of this approach, which is summarized in Figure 2.11.

Conceptually, feature subset selection is a search over all possible subsets of features. Many different types of search strategies can be used, but the search strategy should be computationally inexpensive and should find optimal or near optimal sets of features. It is usually not possible to satisfy both requirements, and thus, trade-offs are necessary.

An integral part of the search is an evaluation step to judge how the current subset of features compares to others that have been considered. This requires an evaluation measure that attempts to determine the goodness of a subset of attributes with respect to a particular data mining task, such as

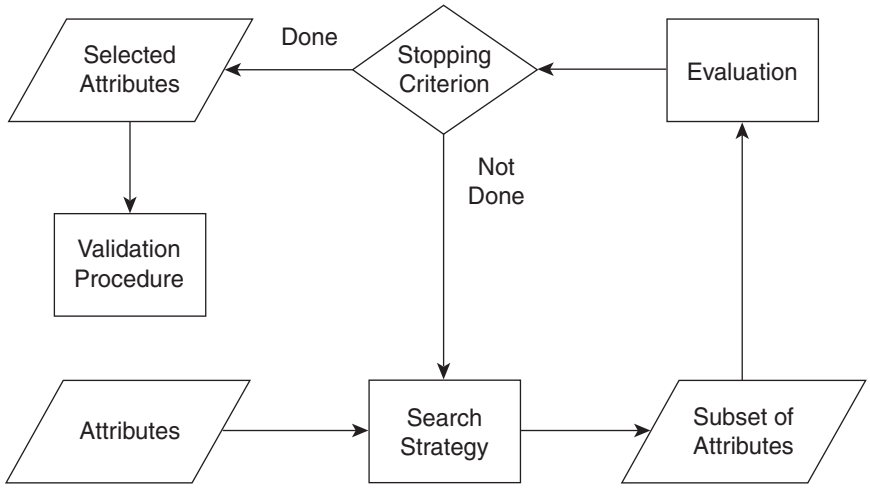


Figure 2.11. Flowchart of a feature subset selection process.

classification or clustering. For the filter approach, such measures attempt to predict how well the actual data mining algorithm will perform on a given set of attributes. For the wrapper approach, where evaluation consists of actually running the target data mining algorithm, the subset evaluation function is simply the criterion normally used to measure the result of the data mining.

Because the number of subsets can be enormous and it is impractical to examine them all, some sort of stopping criterion is necessary. This strategy is usually based on one or more conditions involving the following: the number of iterations, whether the value of the subset evaluation measure is optimal or exceeds a certain threshold, whether a subset of a certain size has been obtained, and whether any improvement can be achieved by the options available to the search strategy.

Finally, once a subset of features has been selected, the results of the target data mining algorithm on the selected subset should be validated. A straightforward validation approach is to run the algorithm with the full set of features and compare the full results to results obtained using the subset of features. Hopefully, the subset of features will produce results that are better than or almost as good as those produced when using all features. Another validation approach is to use a number of different feature selection algorithms to obtain subsets of features and then compare the results of running the data mining algorithm on each subset.

Feature Weighting

Feature weighting is an alternative to keeping or eliminating features. More important features are assigned a higher weight, while less important features are given a lower weight. These weights are sometimes assigned based on domain knowledge about the relative importance of features. Alternatively, they can sometimes be determined automatically. For example, some classification schemes, such as support vector machines (Chapter 6), produce classification models in which each feature is given a weight. Features with larger weights play a more important role in the model. The normalization of objects that takes place when computing the cosine similarity (Section 2.4.5) can also be regarded as a type of feature weighting.

2.3.5 Feature Creation

It is frequently possible to create, from the original attributes, a new set of attributes that captures the important information in a data set much more effectively. Furthermore, the number of new attributes can be smaller than the number of original attributes, allowing us to reap all the previously described benefits of dimensionality reduction. Two related methodologies for creating new attributes are described next: feature extraction and mapping the data to a new space.

Feature Extraction

The creation of a new set of features from the original raw data is known as **feature extraction**. Consider a set of photographs, where each photograph is to be classified according to whether it contains a human face. The raw data is a set of pixels, and as such, is not suitable for many types of classification algorithms. However, if the data is processed to provide higher-level features, such as the presence or absence of certain types of edges and areas that are highly correlated with the presence of human faces, then a much broader set of classification techniques can be applied to this problem.

Unfortunately, in the sense in which it is most commonly used, feature extraction is highly domain-specific. For a particular field, such as image processing, various features and the techniques to extract them have been developed over a period of time, and often these techniques have limited applicability to other fields. Consequently, whenever data mining is applied to a relatively new area, a key task is the development of new features and feature extraction methods.

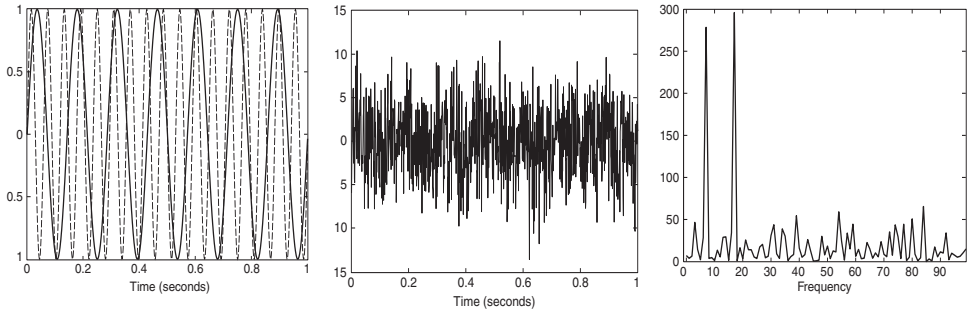
Although feature extraction is often complicated, Example 2.10 illustrates that it can be relatively straightforward.

Example 2.10 (Density)**.** Consider a data set consisting of information about historical artifacts, which, along with other information, contains the volume and mass of each artifact. For simplicity, assume that these artifacts are made of a small number of materials (wood, clay, bronze, gold) and that we want to classify the artifacts with respect to the material of which they are made. In this case, a density feature constructed from the mass and volume features, i.e., $density = mass/volume$, would most directly yield an accurate classification. Although there have been some attempts to automatically perform such simple feature extraction by exploring basic mathematical combinations of existing attributes, the most common approach is to construct features using domain expertise. ■

Mapping the Data to a New Space

A totally different view of the data can reveal important and interesting features. Consider, for example, time series data, which often contains periodic patterns. If there is only a single periodic pattern and not much noise, then the pattern is easily detected. If, on the other hand, there are a number of periodic patterns and a significant amount of noise, then these patterns are hard to detect. Such patterns can, nonetheless, often be detected by applying a **Fourier transform** to the time series in order to change to a representation in which frequency information is explicit. In Example 2.11, it will not be necessary to know the details of the Fourier transform. It is enough to know that, for each time series, the Fourier transform produces a new data object whose attributes are related to frequencies.

Example 2.11 (Fourier Analysis)**.** The time series presented in Figure 2.12(b) is the sum of three other time series, two of which are shown in Figure 2.12(a) and have frequencies of 7 and 17 cycles per second, respectively. The third time series is random noise. Figure 2.12(c) shows the power spectrum that can be computed after applying a Fourier transform to the original time series. (Informally, the power spectrum is proportional to the square of each frequency attribute.) In spite of the noise, there are two peaks that correspond to the periods of the two original, non-noisy time series. Again, the main point is that better features can reveal important aspects of the data. ■



(a) Two time series. (b) Noisy time series. (c) Power spectrum.

Figure 2.12. Application of the Fourier transform to identify the underlying frequencies in time series data.

Many other sorts of transformations are also possible. Besides the Fourier transform, the **wavelet transform** has also proven very useful for time series and other types of data.

2.3.6 Discretization and Binarization

Some data mining algorithms, especially certain classification algorithms, require that the data be in the form of categorical attributes. Algorithms that find association patterns require that the data be in the form of binary attributes. Thus, it is often necessary to transform a continuous attribute into a categorical attribute (**discretization**), and both continuous and discrete attributes may need to be transformed into one or more binary attributes (**binarization**). Additionally, if a categorical attribute has a large number of values (categories), or some values occur infrequently, then it can be beneficial for certain data mining tasks to reduce the number of categories by combining some of the values.

As with feature selection, the best discretization or binarization approach is the one that "produces the best result for the data mining algorithm that will be used to analyze the data." It is typically not practical to apply such a criterion directly. Consequently, discretization or binarization is performed in a way that satisfies a criterion that is thought to have a relationship to good performance for the data mining task being considered. In general, the best discretization depends on the algorithm being used, as well as the other

Table 2.5. Conversion of a categorical attribute to three binary attributes.

Categorical Value	Integer Value	x_1	x_2	x_3
awful	0	0	0	0
poor	1	0	0	1
OK	2	0	1	0
good	3	0	1	1
great	4	1	0	0

Table 2.6. Conversion of a categorical attribute to five asymmetric binary attributes.

Categorical Value	Integer Value	x_1	x_2	x_3	x_4	x_5
awful	0	1	0	0	0	0
poor	1	0	1	0	0	0
OK	2	0	0	1	0	0
good	3	0	0	0	1	0
great	4	0	0	0	0	1

attributes being considered. Typically, however, the discretization of each attribute is considered in isolation.

Binarization

A simple technique to binarize a categorical attribute is the following: If there are m categorical values, then uniquely assign each original value to an integer in the interval $[0, m-1]$. If the attribute is ordinal, then order must be maintained by the assignment. (Note that even if the attribute is originally represented using integers, this process is necessary if the integers are not in the interval $[0, m-1]$.) Next, convert each of these m integers to a binary number. Since $n = \lceil \log_2(m) \rceil$ binary digits are required to represent these integers, represent these binary numbers using n binary attributes. To illustrate, a categorical variable with 5 values {*awful*, *poor*, *OK*, *good*, *great*} would require three binary variables x_1, x_2, and x_3. The conversion is shown in Table 2.5.

Such a transformation can cause complications, such as creating unintended relationships among the transformed attributes. For example, in Table 2.5, attributes x_2 and x_3 are correlated because information about the *good* value is encoded using both attributes. Furthermore, association analysis requires asymmetric binary attributes, where only the presence of the attribute (value = 1) is important. For association problems, it is therefore necessary to introduce one asymmetric binary attribute for each categorical value, as

shown in Table 2.6. If the number of resulting attributes is too large, then the techniques described in the following sections can be used to reduce the number of categorical values before binarization.

Likewise, for association problems, it can be necessary to replace a single binary attribute with two asymmetric binary attributes. Consider a binary attribute that records a person's gender, male or female. For traditional association rule algorithms, this information needs to be transformed into two asymmetric binary attributes, one that is a 1 only when the person is male and one that is a 1 only when the person is female. (For asymmetric binary attributes, the information representation is somewhat inefficient in that two bits of storage are required to represent each bit of information.)

Discretization of Continuous Attributes

Discretization is typically applied to attributes that are used in classification or association analysis. Transformation of a continuous attribute to a categorical attribute involves two subtasks: deciding how many categories, n, to have and determining how to map the values of the continuous attribute to these categories. In the first step, after the values of the continuous attribute are sorted, they are then divided into n intervals by specifying $n-1$ **split points**. In the second, rather trivial step, all the values in one interval are mapped to the same categorical value. Therefore, the problem of discretization is one of deciding how many split points to choose and where to place them. The result can be represented either as a set of intervals $\{(x_0, x_1], (x_1, x_2], \ldots, (x_{n-1}, x_n)\}$, where x_0 and x_n can be $+\infty$ or $-\infty$, respectively, or equivalently, as a series of inequalities $x_0 < x \leq x_1, \ldots, x_{n-1} < x < x_n$.

Unsupervised Discretization A basic distinction between discretization methods for classification is whether class information is used (supervised) or not (unsupervised). If class information is not used, then relatively simple approaches are common. For instance, the **equal width** approach divides the range of the attribute into a user-specified number of intervals each having the same width. Such an approach can be badly affected by outliers, and for that reason, an **equal frequency (equal depth)** approach, which tries to put the same number of objects into each interval, is often preferred. As another example of unsupervised discretization, a clustering method, such as K-means (see Chapter 5), can also be used. Finally, visually inspecting the data can sometimes be an effective approach.

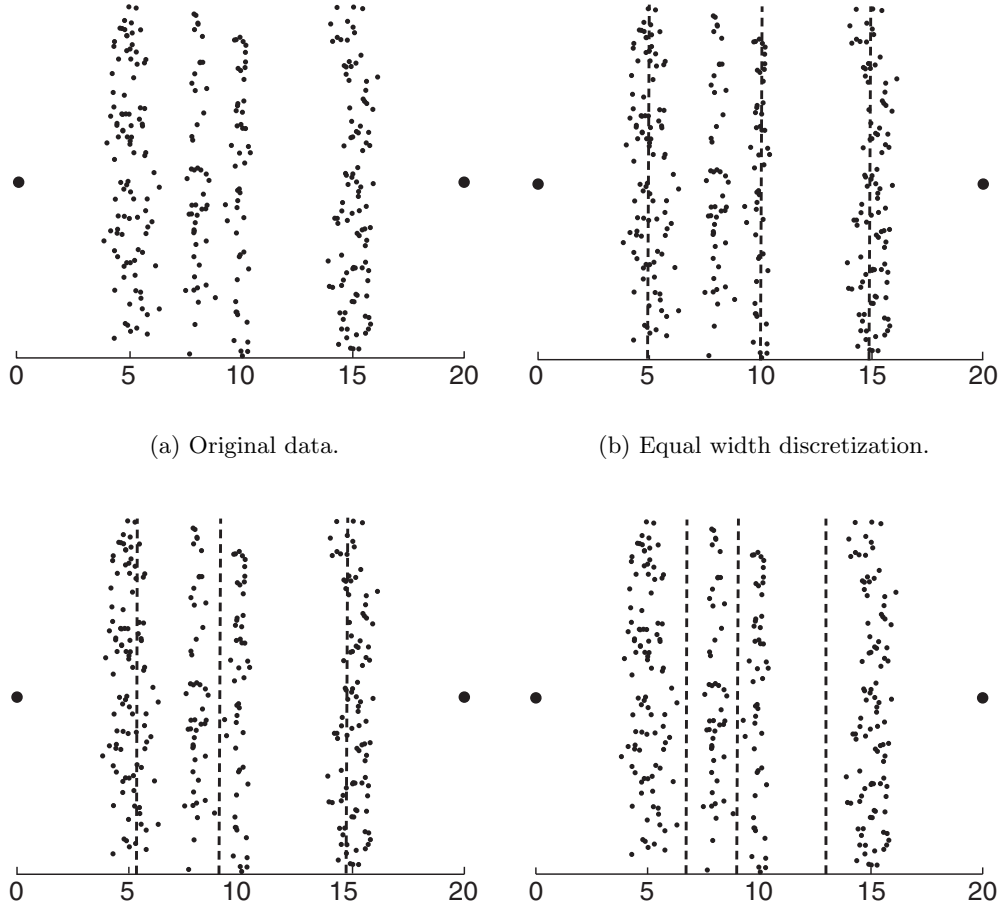
Example 2.12 (Discretization Techniques)**.** This example demonstrates how these approaches work on an actual data set. Figure 2.13(a) shows data points belonging to four different groups, along with two outliers—the large dots on either end. The techniques of the previous paragraph were applied to discretize the x values of these data points into four categorical values. (Points in the data set have a random y component to make it easy to see how many points are in each group.) Visually inspecting the data works quite well, but is not automatic, and thus, we focus on the other three approaches. The split points produced by the techniques equal width, equal frequency, and K-means are shown in Figures 2.13(b), 2.13(c), and 2.13(d), respectively. The split points are represented as dashed lines.

In this particular example, if we measure the performance of a discretization technique by the extent to which different objects that clump together have the same categorical value, then K-means performs best, followed by equal frequency, and finally, equal width. More generally, the best discretization will depend on the application and often involves domain-specific discretization. For example, the discretization of people into low income, middle income, and high income is based on economic factors.

■

Supervised Discretization If classification is our application and class labels are known for some data objects, then discretization approaches that use class labels often produce better classification. This should not be surprising, since an interval constructed with no knowledge of class labels often contains a mixture of class labels. A conceptually simple approach is to place the splits in a way that maximizes the purity of the intervals, i.e., the extent to which an interval contains a single class label. In practice, however, such an approach requires potentially arbitrary decisions about the purity of an interval and the minimum size of an interval.

To overcome such concerns, some statistically based approaches start with each attribute value in a separate interval and create larger intervals by merging adjacent intervals that are similar according to a statistical test. An alternative to this bottom-up approach is a top-down approach that starts by bisecting the initial values so that the resulting two intervals give minimum entropy. This technique only needs to consider each value as a possible split point, because it is assumed that intervals contain ordered sets of values. The splitting process is then repeated with another interval, typically choosing the interval with the worst (highest) entropy, until a user-specified number of intervals is reached, or a stopping criterion is satisfied.



(a) Original data.

(b) Equal width discretization.

(c) Equal frequency discretization.

(d) K-means discretization.

Figure 2.13. Different discretization techniques.

Entropy-based approaches are one of the most promising approaches to discretization, whether bottom-up or top-down. First, it is necessary to define **entropy**. Let k be the number of different class labels, m_i be the number of values in the ith interval of a partition, and m_{ij} be the number of values of class j in interval i. Then the entropy e_i of the ith interval is given by the equation

$$e_i = -\sum_{j=1}^{k} p_{ij} \log_2 p_{ij},$$

where $p_{ij} = m_{ij}/m_i$ is the probability (fraction of values) of class j in the ith interval. The total entropy, e, of the partition is the weighted average of the individual interval entropies, i.e.,

$$e = \sum_{i=1}^{n} w_i e_i,$$

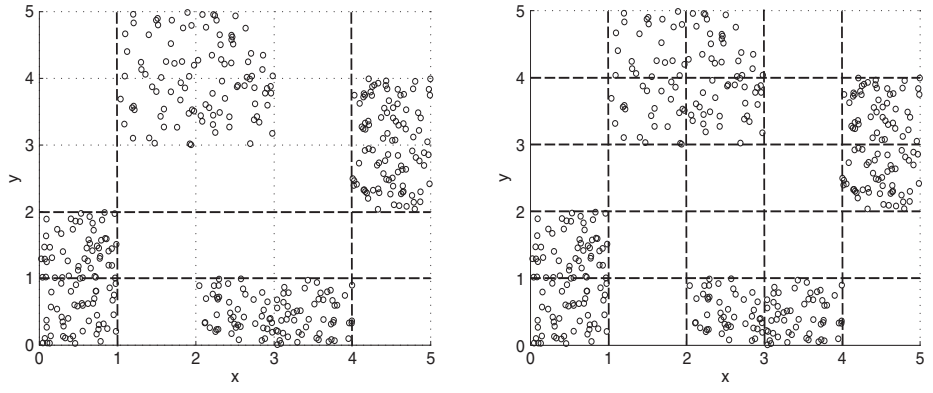
where m is the number of values, $w_i = m_i/m$ is the fraction of values in the ith interval, and n is the number of intervals. Intuitively, the entropy of an interval is a measure of the purity of an interval. If an interval contains only values of one class (is perfectly pure), then the entropy is 0 and it contributes nothing to the overall entropy. If the classes of values in an interval occur equally often (the interval is as impure as possible), then the entropy is a maximum.

Example 2.13 (Discretization of Two Attributes)**.** The top-down method based on entropy was used to independently discretize both the x and y attributes of the two-dimensional data shown in Figure 2.14. In the first discretization, shown in Figure 2.14(a), the x and y attributes were both split into three intervals. (The dashed lines indicate the split points.) In the second discretization, shown in Figure 2.14(b), the x and y attributes were both split into five intervals. ■

This simple example illustrates two aspects of discretization. First, in two dimensions, the classes of points are well separated, but in one dimension, this is not so. In general, discretizing each attribute separately often guarantees suboptimal results. Second, five intervals work better than three, but six intervals do not improve the discretization much, at least in terms of entropy. (Entropy values and results for six intervals are not shown.) Consequently, it is desirable to have a stopping criterion that automatically finds the right number of partitions.

Categorical Attributes with Too Many Values

Categorical attributes can sometimes have too many values. If the categorical attribute is an ordinal attribute, then techniques similar to those for continuous attributes can be used to reduce the number of categories. If the categorical attribute is nominal, however, then other approaches are needed. Consider a university that has a large number of departments. Consequently,



(a) Three intervals (b) Five intervals

Figure 2.14. Discretizing x and y attributes for four groups (classes) of points.

a *department name* attribute might have dozens of different values. In this situation, we could use our knowledge of the relationships among different departments to combine departments into larger groups, such as *engineering*, *social sciences*, or *biological sciences*. If domain knowledge does not serve as a useful guide or such an approach results in poor classification performance, then it is necessary to use a more empirical approach, such as grouping values together only if such a grouping results in improved classification accuracy or achieves some other data mining objective.

2.3.7 Variable Transformation

A **variable transformation** refers to a transformation that is applied to all the values of a variable. (We use the term variable instead of attribute to adhere to common usage, although we will also refer to attribute transformation on occasion.) In other words, for each object, the transformation is applied to the value of the variable for that object. For example, if only the magnitude of a variable is important, then the values of the variable can be transformed by taking the absolute value. In the following section, we discuss two important types of variable transformations: simple functional transformations and normalization.

Simple Functions

For this type of variable transformation, a simple mathematical function is applied to each value individually. If x is a variable, then examples of such transformations include x^k, $\log x$, e^x, $\sqrt{x}$, $1/x$, $\sin x$, or $|x|$. In statistics, variable transformations, especially sqrt, log, and $1/x$, are often used to transform data that does not have a Gaussian (normal) distribution into data that does. While this can be important, other reasons often take precedence in data mining. Suppose the variable of interest is the number of data bytes in a session, and the number of bytes ranges from 1 to 1 billion. This is a huge range, and it can be advantageous to compress it by using a $\log_{10}$ transformation. In this case, sessions that transferred 10^8 and 10^9 bytes would be more similar to each other than sessions that transferred 10 and 1000 bytes ($9 - 8 = 1$ versus $3 - 1 = 2$). For some applications, such as network intrusion detection, this may be what is desired, since the first two sessions most likely represent transfers of large files, while the latter two sessions could be two quite distinct types of sessions.

Variable transformations should be applied with caution because they change the nature of the data. While this is what is desired, there can be problems if the nature of the transformation is not fully appreciated. For instance, the transformation $1/x$ reduces the magnitude of values that are 1 or larger, but increases the magnitude of values between 0 and 1. To illustrate, the values $\{1, 2, 3\}$ go to $\{1, \frac{1}{2}, \frac{1}{3}\}$, but the values $\{1, \frac{1}{2}, \frac{1}{3}\}$ go to $\{1, 2, 3\}$. Thus, for all sets of values, the transformation $1/x$ reverses the order. To help clarify the effect of a transformation, it is important to ask questions such as the following: What is the desired property of the transformed attribute? Does the order need to be maintained? Does the transformation apply to all values, especially negative values and 0? What is the effect of the transformation on the values between 0 and 1? Exercise 21 on page 129 explores other aspects of variable transformation.

Normalization or Standardization

The goal of standardization or normalization is to make an entire set of values have a particular property. A traditional example is that of "standardizing a variable" in statistics. If $\overline{x}$ is the mean (average) of the attribute values and s_x is their standard deviation, then the transformation $x' = (x - \overline{x})/s_x$ creates a new variable that has a mean of 0 and a standard deviation of 1. If different variables are to be used together, e.g., for clustering, then such a transformation is often necessary to avoid having a variable with large values

dominate the results of the analysis. To illustrate, consider comparing people based on two variables: age and income. For any two people, the difference in income will likely be much higher in absolute terms (hundreds or thousands of dollars) than the difference in age (less than 150). If the differences in the range of values of age and income are not taken into account, then the comparison between people will be dominated by differences in income. In particular, if the similarity or dissimilarity of two people is calculated using the similarity or dissimilarity measures defined later in this chapter, then in many cases, such as that of Euclidean distance, the income values will dominate the calculation.

The mean and standard deviation are strongly affected by outliers, so the above transformation is often modified. First, the mean is replaced by the **median**, i.e., the middle value. Second, the standard deviation is replaced by the **absolute standard deviation**. Specifically, if x is a variable, then the absolute standard deviation of x is given by $\sigma_A = \sum_{i=1}^{m} |x_i - \mu|$, where x_i is the ith value of the variable, m is the number of objects, and μ is either the mean or median. Other approaches for computing estimates of the location (center) and spread of a set of values in the presence of outliers are described in statistics books. These more robust measures can also be used to define a standardization transformation.

2.4 Measures of Similarity and Dissimilarity

Similarity and dissimilarity are important because they are used by a number of data mining techniques, such as clustering, nearest neighbor classification, and anomaly detection. In many cases, the initial data set is not needed once these similarities or dissimilarities have been computed. Such approaches can be viewed as transforming the data to a similarity (dissimilarity) space and then performing the analysis. Indeed, **kernel methods** are a powerful realization of this idea. These methods are introduced in Section 2.4.7 and are discussed more fully in the context of classification in Section 6.9.4.

We begin with a discussion of the basics: high-level definitions of similarity and dissimilarity, and a discussion of how they are related. For convenience, the term **proximity** is used to refer to either similarity or dissimilarity. Since the proximity between two objects is a function of the proximity between the corresponding attributes of the two objects, we first describe how to measure the proximity between objects having only one attribute.

We then consider proximity measures for objects with multiple attributes. This includes measures such as the Jaccard and cosine similarity measures, which are useful for sparse data, such as documents, as well as correlation

and Euclidean distance, which are useful for non-sparse (dense) data, such as time series or multi-dimensional points. We also consider mutual information, which can be applied to many types of data and is good for detecting nonlinear relationships. In this discussion, we restrict ourselves to objects with relatively homogeneous attribute types, typically binary or continuous.

Next, we consider several important issues concerning proximity measures. This includes how to compute proximity between objects when they have heterogeneous types of attributes, and approaches to account for differences of scale and correlation among variables when computing distance between numerical objects. The section concludes with a brief discussion of how to select the right proximity measure.

Although this section focuses on the computation of proximity between data objects, proximity can also be computed between attributes. For example, for the document-term matrix of Figure 2.2(d), the cosine measure can be used to compute similarity between a pair of documents or a pair of terms (words). Knowing that two variables are strongly related can, for example, be helpful for eliminating redundancy. In particular, the correlation and mutual information measures discussed later are often used for that purpose.

2.4.1 Basics

Definitions

Informally, the **similarity** between two objects is a numerical measure of the degree to which the two objects are alike. Consequently, similarities are *higher* for pairs of objects that are more alike. Similarities are usually non-negative and are often between 0 (no similarity) and 1 (complete similarity).

The **dissimilarity** between two objects is a numerical measure of the degree to which the two objects are different. Dissimilarities are *lower* for more similar pairs of objects. Frequently, the term **distance** is used as a synonym for dissimilarity, although, as we shall see, distance often refers to a special class of dissimilarities. Dissimilarities sometimes fall in the interval $[0, 1]$, but it is also common for them to range from 0 to ∞.

Transformations

Transformations are often applied to convert a similarity to a dissimilarity, or vice versa, or to transform a proximity measure to fall within a particular range, such as [0,1]. For instance, we may have similarities that range from 1 to 10, but the particular algorithm or software package that we want to use may be designed to work only with dissimilarities, or it may work only with

similarities in the interval [0,1]. We discuss these issues here because we will employ such transformations later in our discussion of proximity. In addition, these issues are relatively independent of the details of specific proximity measures.

Frequently, proximity measures, especially similarities, are defined or transformed to have values in the interval [0,1]. Informally, the motivation for this is to use a scale in which a proximity value indicates the fraction of similarity (or dissimilarity) between two objects. Such a transformation is often relatively straightforward. For example, if the similarities between objects range from 1 (not at all similar) to 10 (completely similar), we can make them fall within the range $[0, 1]$ by using the transformation $s' = (s - 1)/9$, where s and s' are the original and new similarity values, respectively. In the more general case, the transformation of similarities to the interval $[0, 1]$ is given by the expression $s' = (s - min_s)/(max_s - min_s)$, where max_s and min_s are the maximum and minimum similarity values, respectively. Likewise, dissimilarity measures with a finite range can be mapped to the interval [0,1] by using the formula $d' = (d - min_d)/(max_d - min_d)$. This is an example of a linear transformation, which preserves the relative distances between points. In other words, if points, x_1 and x_2, are twice as far apart as points, x_3 and x_4, the same will be true after a linear transformation.

However, there can be complications in mapping proximity measures to the interval $[0, 1]$ using a linear transformation. If, for example, the proximity measure originally takes values in the interval [0,∞], then max_d is not defined and a nonlinear transformation is needed. Values will not have the same relationship to one another on the new scale. Consider the transformation $d' = d/(1 + d)$ for a dissimilarity measure that ranges from 0 to ∞. The dissimilarities 0, 0.5, 2, 10, 100, and 1000 will be transformed into the new dissimilarities 0, 0.33, 0.67, 0.90, 0.99, and 0.999, respectively. Larger values on the original dissimilarity scale are compressed into the range of values near 1, but whether this is desirable depends on the application.

Note that mapping proximity measures to the interval $[0, 1]$ can also change the meaning of the proximity measure. For example, correlation, which is discussed later, is a measure of similarity that takes values in the interval $[-1, 1]$. Mapping these values to the interval [0,1] by taking the absolute value loses information about the sign, which can be important in some applications. See Exercise 27 on page 131.

Transforming similarities to dissimilarities and vice versa is also relatively straightforward, although we again face the issues of preserving meaning and changing a linear scale into a nonlinear scale. If the similarity (or dissimilarity) falls in the interval [0,1], then the dissimilarity can be defined as $d = 1 - s$

($s = 1 - d$). Another simple approach is to define similarity as the negative of the dissimilarity (or vice versa). To illustrate, the dissimilarities 0, 1, 10, and 100 can be transformed into the similarities 0, -1, -10, and -100, respectively.

The similarities resulting from the negation transformation are not restricted to the range $[0, 1]$, but if that is desired, then transformations such as $s = \frac{1}{d+1}$, $s = e^{-d}$, or $s = 1 - \frac{d - min_d}{max_d - min_d}$ can be used. For the transformation $s = \frac{1}{d+1}$, the dissimilarities 0, 1, 10, 100 are transformed into 1, 0.5, 0.09, 0.01, respectively. For $s = e^{-d}$, they become 1.00, 0.37, 0.00, 0.00, respectively, while for $s = 1 - \frac{d - min_d}{max_d - min_d}$ they become 1.00, 0.99, 0.90, 0.00, respectively. In this discussion, we have focused on converting dissimilarities to similarities. Conversion in the opposite direction is considered in Exercise 28 on page 131.

In general, any monotonic decreasing function can be used to convert dissimilarities to similarities, or vice versa. Of course, other factors also must be considered when transforming similarities to dissimilarities, or vice versa, or when transforming the values of a proximity measure to a new scale. We have mentioned issues related to preserving meaning, distortion of scale, and requirements of data analysis tools, but this list is certainly not exhaustive.

2.4.2 Similarity and Dissimilarity between Simple Attributes

The proximity of objects with a number of attributes is typically defined by combining the proximities of individual attributes, and thus, we first discuss proximity between objects having a single attribute. Consider objects described by one nominal attribute. What would it mean for two such objects to be similar? Because nominal attributes convey only information about the distinctness of objects, all we can say is that two objects either have the same value or they do not. Hence, in this case similarity is traditionally defined as 1 if attribute values match, and as 0 otherwise. A dissimilarity would be defined in the opposite way: 0 if the attribute values match, and 1 if they do not.

For objects with a single ordinal attribute, the situation is more complicated because information about order should be taken into account. Consider an attribute that measures the quality of a product, e.g., a candy bar, on the scale {*poor*, *fair*, *OK*, *good*, *wonderful*}. It would seem reasonable that a product, P1, which is rated *wonderful*, would be closer to a product P2, which is rated *good*, than it would be to a product P3, which is rated *OK*. To make this observation quantitative, the values of the ordinal attribute are often mapped to successive integers, beginning at 0 or 1, e.g., {poor=0, fair=1, OK=2, $good = 3$, $wonderful = 4$}. Then, $d(\text{P1}, \text{P2}) = 3 - 2 = 1$ or, if we want the dissimilarity to fall between 0 and 1, $d(\text{P1}, \text{P2}) = \frac{3-2}{4} = 0.25$. A similarity for ordinal attributes can then be defined as $s = 1 - d$.

Table 2.7. Similarity and dissimilarity for simple attributes

Attribute Type	Dissimilarity	Similarity
Nominal	$d = \begin{cases} 0 & \text{if } x = y \\ 1 & \text{if } x \neq y \end{cases}$	$s = \begin{cases} 1 & \text{if } x = y \\ 0 & \text{if } x \neq y \end{cases}$
Ordinal	$d = \lvert x - y \rvert / (n-1)$ (values mapped to integers 0 to $n-1$, where n is the number of values)	$s = 1 - d$
Interval or Ratio	$d = \lvert x - y \rvert$	$s = -d$, $s = \frac{1}{1+d}$, $s = e^{-d}$, $s = 1 - \frac{d - min_d}{max_d - min_d}$

This definition of similarity (dissimilarity) for an ordinal attribute should make the reader a bit uneasy since this assumes equal intervals between successive values of the attribute, and this is not necessarily so. Otherwise, we would have an interval or ratio attribute. Is the difference between the values *fair* and *good* really the same as that between the values *OK* and *wonderful*? Probably not, but in practice, our options are limited, and in the absence of more information, this is the standard approach for defining proximity between ordinal attributes.

For interval or ratio attributes, the natural measure of dissimilarity between two objects is the absolute difference of their values. For example, we might compare our current weight and our weight a year ago by saying "I am ten pounds heavier." In cases such as these, the dissimilarities typically range from 0 to ∞, rather than from 0 to 1. The similarity of interval or ratio attributes is typically expressed by transforming a dissimilarity into a similarity, as previously described.

Table 2.7 summarizes this discussion. In this table, x and y are two objects that have one attribute of the indicated type. Also, $d(x, y)$ and $s(x, y)$ are the dissimilarity and similarity between x and y, respectively. Other approaches are possible; these are the most common ones.

The following two sections consider more complicated measures of proximity between objects that involve multiple attributes: (1) dissimilarities between data objects and (2) similarities between data objects. This division allows us to more naturally display the underlying motivations for employing various proximity measures. We emphasize, however, that similarities can be transformed into dissimilarities and vice versa using the approaches described earlier.

2.4.3 Dissimilarities between Data Objects

In this section, we discuss various kinds of dissimilarities. We begin with a discussion of distances, which are dissimilarities with certain properties, and then provide examples of more general kinds of dissimilarities.

Distances

We first present some examples, and then offer a more formal description of distances in terms of the properties common to all distances. The **Euclidean distance**, d, between two points, $\mathbf{x}$ and $\mathbf{y}$, in one-, two-, three-, or higher-dimensional space, is given by the following familiar formula:

$$d(\mathbf{x}, \mathbf{y}) = \sqrt{\sum_{k=1}^{n} (x_k - y_k)^2}, \tag{2.1}$$

where n is the number of dimensions and x_k and y_k are, respectively, the kth attributes (components) of $\mathbf{x}$ and $\mathbf{y}$. We illustrate this formula with Figure 2.15 and Tables 2.8 and 2.9, which show a set of points, the x and y coordinates of these points, and the **distance matrix** containing the pairwise distances of these points.

The Euclidean distance measure given in Equation 2.1 is generalized by the **Minkowski** distance metric shown in Equation 2.2,

$$d(\mathbf{x}, \mathbf{y}) = \left(\sum_{k=1}^{n} |x_k - y_k|^r \right)^{1/r}, \tag{2.2}$$

where r is a parameter. The following are the three most common examples of Minkowski distances.

- $r = 1$. City block (Manhattan, taxicab, L_1 norm) distance. A common example is the **Hamming distance**, which is the number of bits that is different between two objects that have only binary attributes, i.e., between two binary vectors.
- $r = 2$. Euclidean distance (L_2 norm).
- $r = \infty$. Supremum (L_{max} or L_∞ norm) distance. This is the maximum difference between any attribute of the objects. More formally, the L_∞

distance is defined by Equation 2.3

$$d(\mathbf{x}, \mathbf{y}) = \lim_{r \to \infty} \left(\sum_{k=1}^{n} |x_k - y_k|^r \right)^{1/r}. \tag{2.3}$$

The r parameter should not be confused with the number of dimensions (attributes) n. The Euclidean, Manhattan, and supremum distances are defined for all values of n: $1, 2, 3, \ldots$, and specify different ways of combining the differences in each dimension (attribute) into an overall distance.

Tables 2.10 and 2.11, respectively, give the proximity matrices for the L_1 and L_∞ distances using data from Table 2.8. Notice that all these distance matrices are symmetric; i.e., the ij^{th} entry is the same as the ji^{th} entry. In Table 2.9, for instance, the fourth row of the first column and the fourth column of the first row both contain the value 5.1.

Distances, such as the Euclidean distance, have some well-known properties. If $d(\mathbf{x}, \mathbf{y})$ is the distance between two points, $\mathbf{x}$ and $\mathbf{y}$, then the following properties hold.

1. **Positivity**
   (a) $d(\mathbf{x}, \mathbf{y}) \geq 0$ for all $\mathbf{x}$ and $\mathbf{y}$,
   (b) $d(\mathbf{x}, \mathbf{y}) = 0$ only if $\mathbf{x} = \mathbf{y}$.
2. **Symmetry**
   $d(\mathbf{x}, \mathbf{y}) = d(\mathbf{y}, \mathbf{x})$ for all $\mathbf{x}$ and $\mathbf{y}$.
3. **Triangle Inequality**
   $d(\mathbf{x}, \mathbf{z}) \leq d(\mathbf{x}, \mathbf{y}) + d(\mathbf{y}, \mathbf{z})$ for all points $\mathbf{x}$, $\mathbf{y}$, and $\mathbf{z}$.

Measures that satisfy all three properties are known as **metrics**. Some people use the term distance only for dissimilarity measures that satisfy these properties, but that practice is often violated. The three properties described here are useful, as well as mathematically pleasing. Also, if the triangle inequality holds, then this property can be used to increase the efficiency of techniques (including clustering) that depend on distances possessing this property. (See Exercise 30.) Nonetheless, many dissimilarities do not satisfy one or more of the metric properties. Example 2.14 illustrates such a measure.

Example 2.14 (Non-metric Dissimilarities: Set Differences)**.** This example is based on the notion of the difference of two sets, as defined in set theory. Given two sets A and B, $A - B$ is the set of elements of A that are not in

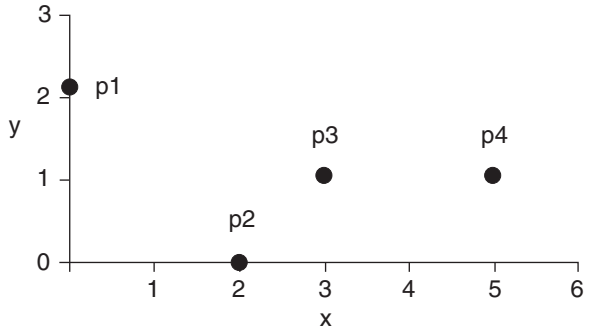


Figure 2.15. Four two-dimensional points.

Table 2.8. x and y coordinates of four points.

point	x coordinate	y coordinate
p1	0	2
p2	2	0
p3	3	1
p4	5	1

Table 2.9. Euclidean distance matrix for Table 2.8.

	p1	p2	p3	p4
p1	0.0	2.8	3.2	5.1
p2	2.8	0.0	1.4	3.2
p3	3.2	1.4	0.0	2.0
p4	5.1	3.2	2.0	0.0

Table 2.10. L_1 distance matrix for Table 2.8.

L_1	p1	p2	p3	p4
p1	0.0	4.0	4.0	6.0
p2	4.0	0.0	2.0	4.0
p3	4.0	2.0	0.0	2.0
p4	6.0	4.0	2.0	0.0

Table 2.11. L_∞ distance matrix for Table 2.8.

L_∞	p1	p2	p3	p4
p1	0.0	2.0	3.0	5.0
p2	2.0	0.0	1.0	3.0
p3	3.0	1.0	0.0	2.0
p4	5.0	3.0	2.0	0.0

B. For example, if $A = \{1, 2, 3, 4\}$ and $B = \{2, 3, 4\}$, then $A - B = \{1\}$ and $B - A = \emptyset$, the empty set. We can define the distance d between two sets A and B as $d(A, B) = size(A - B)$, where $size$ is a function returning the number of elements in a set. This distance measure, which is an integer value greater than or equal to 0, does not satisfy the second part of the positivity property, the symmetry property, or the triangle inequality. However, these properties can be made to hold if the dissimilarity measure is modified as follows: $d(A, B) = size(A-B)+size(B-A)$. See Exercise 26 on page 131. ■

2.4.4 Similarities between Data Objects

For similarities, the triangle inequality (or the analogous property) typically does not hold, but symmetry and positivity typically do. To be explicit, if

$s(\mathbf{x}, \mathbf{y})$ is the similarity between points $\mathbf{x}$ and $\mathbf{y}$, then the typical properties of similarities are the following:

1. $s(\mathbf{x}, \mathbf{y}) = 1$ only if $\mathbf{x} = \mathbf{y}$. $(0 \leq s \leq 1)$
2. $s(\mathbf{x}, \mathbf{y}) = s(\mathbf{y}, \mathbf{x})$ for all $\mathbf{x}$ and $\mathbf{y}$. (Symmetry)

There is no general analog of the triangle inequality for similarity measures. It is sometimes possible, however, to show that a similarity measure can easily be converted to a metric distance. The cosine and Jaccard similarity measures, which are discussed shortly, are two examples. Also, for specific similarity measures, it is possible to derive mathematical bounds on the similarity between two objects that are similar in spirit to the triangle inequality.

Example 2.15 (A Non-symmetric Similarity Measure)**.** Consider an experiment in which people are asked to classify a small set of characters as they flash on a screen. The **confusion matrix** for this experiment records how often each character is classified as itself, and how often each is classified as another character. Using the confusion matrix, we can define a similarity measure between a character x and a character y as the number of times that x is misclassified as y, but note that this measure is not symmetric. For example, suppose that "0" appeared 200 times and was classified as a "0" 160 times, but as an "o" 40 times. Likewise, suppose that "o" appeared 200 times and was classified as an "o" 170 times, but as "0" only 30 times. Then, s(0,o) = 40, but s(o, 0) = 30. In such situations, the similarity measure can be made symmetric by setting $s'(x, y) = s'(y, x) = (s(x, y) + s(y, x))/2$, where s' indicates the new similarity measure. ■

2.4.5 Examples of Proximity Measures

This section provides specific examples of some similarity and dissimilarity measures.

Similarity Measures for Binary Data

Similarity measures between objects that contain only binary attributes are called **similarity coefficients**, and typically have values between 0 and 1. A value of 1 indicates that the two objects are completely similar, while a value of 0 indicates that the objects are not at all similar. There are many rationales for why one coefficient is better than another in specific instances.

Let $\mathbf{x}$ and $\mathbf{y}$ be two objects that consist of n binary attributes. The comparison of two such objects, i.e., two binary vectors, leads to the following

four quantities (frequencies):

f_{00} = the number of attributes where **x** is 0 and **y** is 0
f_{01} = the number of attributes where **x** is 0 and **y** is 1
f_{10} = the number of attributes where **x** is 1 and **y** is 0
f_{11} = the number of attributes where **x** is 1 and **y** is 1

Simple Matching Coefficient One commonly used similarity coefficient is the **simple matching coefficient** (SMC), which is defined as

$$SMC = \frac{\text{number of matching attribute values}}{\text{number of attributes}} = \frac{f_{11} + f_{00}}{f_{01} + f_{10} + f_{11} + f_{00}}. \quad (2.4)$$

This measure counts both presences and absences equally. Consequently, the SMC could be used to find students who had answered questions similarly on a test that consisted only of true/false questions.

Jaccard Coefficient Suppose that **x** and **y** are data objects that represent two rows (two transactions) of a transaction matrix (see Section 2.1.2). If each asymmetric binary attribute corresponds to an item in a store, then a 1 indicates that the item was purchased, while a 0 indicates that the product was not purchased. Because the number of products not purchased by any customer far outnumbers the number of products that were purchased, a similarity measure such as SMC would say that all transactions are very similar. As a result, the Jaccard coefficient is frequently used to handle objects consisting of asymmetric binary attributes. The **Jaccard coefficient**, which is often symbolized by J, is given by the following equation:

$$J = \frac{\text{number of matching presences}}{\text{number of attributes not involved in 00 matches}} = \frac{f_{11}}{f_{01} + f_{10} + f_{11}}. \quad (2.5)$$

Example 2.16 (The SMC and Jaccard Similarity Coefficients)**.** To illustrate the difference between these two similarity measures, we calculate SMC and J for the following two binary vectors.

$\mathbf{x} = (1, 0, 0, 0, 0, 0, 0, 0, 0, 0)$
$\mathbf{y} = (0, 0, 0, 0, 0, 0, 1, 0, 0, 1)$

$f_{01} = 2$ the number of attributes where **x** was 0 and **y** was 1

$f_{10} = 1$ the number of attributes where $\mathbf{x}$ was 1 and $\mathbf{y}$ was 0
$f_{00} = 7$ the number of attributes where $\mathbf{x}$ was 0 and $\mathbf{y}$ was 0
$f_{11} = 0$ the number of attributes where $\mathbf{x}$ was 1 and $\mathbf{y}$ was 1

$$SMC = \frac{f_{11} + f_{00}}{f_{01} + f_{10} + f_{11} + f_{00}} = \frac{0 + 7}{2 + 1 + 0 + 7} = 0.7$$

$$J = \frac{f_{11}}{f_{01} + f_{10} + f_{11}} = \frac{0}{2 + 1 + 0} = 0$$

■

Cosine Similarity

Documents are often represented as vectors, where each component (attribute) represents the frequency with which a particular term (word) occurs in the document. Even though documents have thousands or tens of thousands of attributes (terms), each document is sparse since it has relatively few non-zero attributes. Thus, as with transaction data, similarity should not depend on the number of shared 0 values because any two documents are likely to "not contain" many of the same words, and therefore, if 0–0 matches are counted, most documents will be highly similar to most other documents. Therefore, a similarity measure for documents needs to ignores 0–0 matches like the Jaccard measure, but also must be able to handle non-binary vectors. The **cosine similarity**, defined next, is one of the most common measures of document similarity. If $\mathbf{x}$ and $\mathbf{y}$ are two document vectors, then

$$\cos(\mathbf{x}, \mathbf{y}) = \frac{\langle \mathbf{x}, \mathbf{y} \rangle}{\|\mathbf{x}\| \, \|\mathbf{y}\|} = \frac{\mathbf{x}'\mathbf{y}}{\|\mathbf{x}\| \, \|\mathbf{y}\|}, \tag{2.6}$$

where $'$ indicates vector or matrix transpose and $\langle \mathbf{x}, \mathbf{y} \rangle$ indicates the inner product of the two vectors,

$$\langle \mathbf{x}, \mathbf{y} \rangle = \sum_{k=1}^{n} x_k y_k = \mathbf{x}'\mathbf{y}, \tag{2.7}$$

and $\|\mathbf{x}\|$ is the length of vector $\mathbf{x}$, $\|\mathbf{x}\| = \sqrt{\sum_{k=1}^{n} x_k^2} = \sqrt{\langle \mathbf{x}, \mathbf{x} \rangle} = \sqrt{\mathbf{x}'\mathbf{x}}$.

The inner product of two vectors works well for asymmetric attributes since it depends only on components that are non-zero in both vectors. Hence, the similarity between two documents depends only upon the words that appear in both of them.

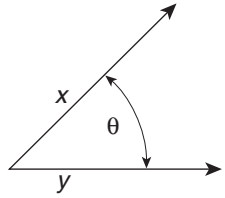


Figure 2.16. Geometric illustration of the cosine measure.

Example 2.17 (Cosine Similarity between Two Document Vectors)**.** This example calculates the cosine similarity for the following two data objects, which might represent document vectors:

$\mathbf{x} = (3, 2, 0, 5, 0, 0, 0, 2, 0, 0)$
$\mathbf{y} = (1, 0, 0, 0, 0, 0, 0, 1, 0, 2)$

$\langle \mathbf{x}, \mathbf{y} \rangle = 3 \times 1 + 2 \times 0 + 0 \times 0 + 5 \times 0 + 0 \times 0 + 0 \times 0 + 0 \times 0 + 2 \times 1 + 0 \times 0 + 0 \times 2 = 5$
$\|\mathbf{x}\| = \sqrt{3 \times 3 + 2 \times 2 + 0 \times 0 + 5 \times 5 + 0 \times 0 + 0 \times 0 + 0 \times 0 + 2 \times 2 + 0 \times 0 + 0 \times 0} = 6.48$
$\|\mathbf{y}\| = \sqrt{1 \times 1 + 0 \times 0 + 0 \times 0 + 0 \times 0 + 0 \times 0 + 0 \times 0 + 0 \times 0 + 1 \times 1 + 0 \times 0 + 2 \times 2} = 2.45$
$\mathbf{cos}(\mathbf{x}, \mathbf{y}) = \mathbf{0.31}$

■

As indicated by Figure 2.16, cosine similarity really is a measure of the (cosine of the) angle between $\mathbf{x}$ and $\mathbf{y}$. Thus, if the cosine similarity is 1, the angle between $\mathbf{x}$ and $\mathbf{y}$ is 0°, and $\mathbf{x}$ and $\mathbf{y}$ are the same except for length. If the cosine similarity is 0, then the angle between $\mathbf{x}$ and $\mathbf{y}$ is 90°, and they do not share any terms (words).

Equation 2.6 also can be written as Equation 2.8.

$$\cos(\mathbf{x}, \mathbf{y}) = \left\langle \frac{\mathbf{x}}{\|\mathbf{x}\|}, \frac{\mathbf{y}}{\|\mathbf{y}\|} \right\rangle = \langle \mathbf{x}', \mathbf{y}' \rangle, \tag{2.8}$$

where $\mathbf{x}' = \mathbf{x}/\|\mathbf{x}\|$ and $\mathbf{y}' = \mathbf{y}/\|\mathbf{y}\|$. Dividing $\mathbf{x}$ and $\mathbf{y}$ by their lengths normalizes them to have a length of 1. This means that cosine similarity does not take the *length* of the two data objects into account when computing similarity. (Euclidean distance might be a better choice when length is important.) For vectors with a length of 1, the cosine measure can be calculated by taking a simple inner product. Consequently, when many cosine similarities between objects are being computed, normalizing the objects to have unit length can reduce the time required.

Extended Jaccard Coefficient (Tanimoto Coefficient)

The extended Jaccard coefficient can be used for document data and that reduces to the Jaccard coefficient in the case of binary attributes. This coefficient, which we shall represent as EJ, is defined by the following equation:

$$EJ(\mathbf{x}, \mathbf{y}) = \frac{\langle \mathbf{x}, \mathbf{y} \rangle}{\|\mathbf{x}\|^2 + \|\mathbf{y}\|^2 - \langle \mathbf{x}, \mathbf{y} \rangle} = \frac{\mathbf{x}'\mathbf{y}}{\|\mathbf{x}\|^2 + \|\mathbf{y}\|^2 - \mathbf{x}'\mathbf{y}}. \tag{2.9}$$

Correlation

Correlation is frequently used to measure the linear relationship between two sets of values that are observed together. Thus, correlation can measure the relationship between two variables (height and weight) or between two objects (a pair of temperature time series). Correlation is used much more frequently to measure the similarity between attributes since the values in two data objects come from different attributes, which can have very different attribute types and scales. There are many types of correlation, and indeed correlation is sometimes used in a general sense to mean the relationship between two sets of values that are observed together. In this discussion, we will focus on a measure appropriate for numerical values.

Specifically, **Pearson's correlation** between two sets of numerical values, i.e., two vectors, $\mathbf{x}$ and $\mathbf{y}$, is defined by the following equation:

$$\text{corr}(\mathbf{x}, \mathbf{y}) = \frac{\text{covariance}(\mathbf{x}, \mathbf{y})}{\text{standard_deviation}(\mathbf{x}) \times \text{standard_deviation}(\mathbf{y})} = \frac{s_{xy}}{s_x \; s_y}, \tag{2.10}$$

where we use the following standard statistical notation and definitions:

$$\text{covariance}(\mathbf{x}, \mathbf{y}) = s_{xy} = \frac{1}{n-1} \sum_{k=1}^{n} (x_k - \overline{x})(y_k - \overline{y}) \tag{2.11}$$

$$\text{standard_deviation}(\mathbf{x}) = s_x = \sqrt{\frac{1}{n-1} \sum_{k=1}^{n} (x_k - \overline{x})^2}$$

$$\text{standard_deviation}(\mathbf{y}) = s_y = \sqrt{\frac{1}{n-1} \sum_{k=1}^{n} (y_k - \overline{y})^2}$$

$$\overline{x} = \frac{1}{n}\sum_{k=1}^{n} x_k \text{ is the mean of } \mathbf{x}$$
$$\overline{y} = \frac{1}{n}\sum_{k=1}^{n} y_k \text{ is the mean of } \mathbf{y}$$

Example 2.18 (Perfect Correlation)**.** Correlation is always in the range -1 to 1. A correlation of 1 (-1) means that $\mathbf{x}$ and $\mathbf{y}$ have a perfect positive (negative) linear relationship; that is, $x_k = ay_k + b$, where a and b are constants. The following two vectors $\mathbf{x}$ and $\mathbf{y}$ illustrate cases where the correlation is -1 and $+1$, respectively. In the first case, the means of $\mathbf{x}$ and $\mathbf{y}$ were chosen to be 0, for simplicity.

$\mathbf{x} = (-3,\ \ 6,\ 0,\ \ 3, -6)$
$\mathbf{y} = (\ \ 1, -2,\ 0, -1,\ \ 2)$
$corr(\mathbf{x}, \mathbf{y}) = -1 \quad x_k = -3y_k$

$\mathbf{x} = (3, 6, 0, 3, 6)$
$\mathbf{y} = (1, 2, 0, 1, 2)$
$corr(\mathbf{x}, \mathbf{y}) = 1 \quad x_k = 3y_k$ ■

Example 2.19 (Nonlinear Relationships)**.** If the correlation is 0, then there is no linear relationship between the two sets of values. However, nonlinear relationships can still exist. In the following example, $y_k = x_k^2$, but their correlation is 0.

$\mathbf{x} = (-3, -2, -1,\ 0,\ 1,\ 2,\ 3)$
$\mathbf{y} = (\ \ 9,\ \ \ 4,\ \ \ 1,\ 0,\ 1,\ 4,\ 9)$

■

Example 2.20 (Visualizing Correlation)**.** It is also easy to judge the correlation between two vectors $\mathbf{x}$ and $\mathbf{y}$ by plotting pairs of corresponding values of $\mathbf{x}$ and $\mathbf{y}$ in a scatter plot. Figure 2.17 shows a number of these scatter plots when $\mathbf{x}$ and $\mathbf{y}$ consist of a set of 30 pairs of values that are randomly generated (with a normal distribution) so that the correlation of $\mathbf{x}$ and $\mathbf{y}$ ranges from -1 to 1. Each circle in a plot represents one of the 30 pairs of $\mathbf{x}$ and $\mathbf{y}$ values; its x coordinate is the value of that pair for $\mathbf{x}$, while its y coordinate is the value of the same pair for $\mathbf{y}$. ■

If we transform $\mathbf{x}$ and $\mathbf{y}$ by subtracting off their means and then normalizing them so that their lengths are 1, then their correlation can be calculated by taking the dot product. Let us refer to these transformed vectors of $\mathbf{x}$ and $\mathbf{y}$ as $\mathbf{x}'$ and $\mathbf{y}'$, respectively. (Notice that this transformation is not the same

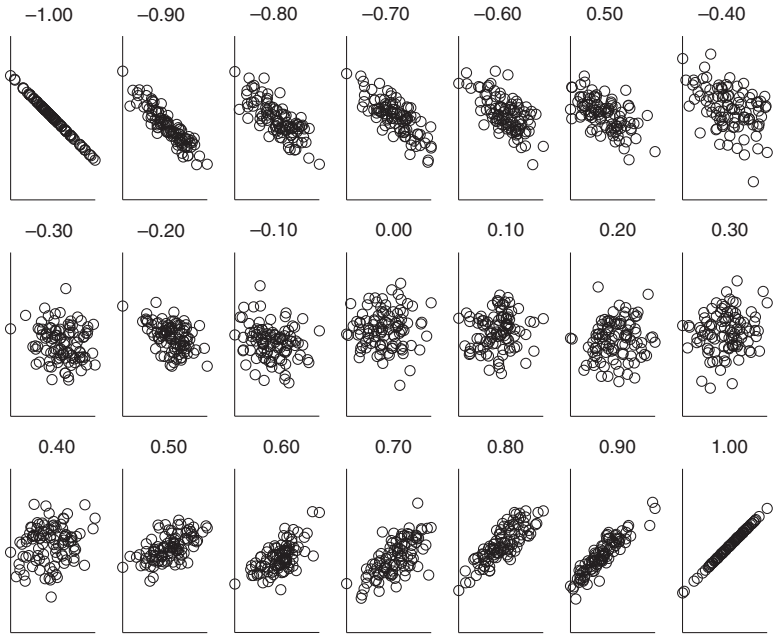


Figure 2.17. Scatter plots illustrating correlations from -1 to 1.

as the standardization used in other contexts, where we subtract the means and divide by the standard deviations, as discussed in Section 2.3.7.) This transformation highlights an interesting relationship between the correlation measure and the cosine measure. Specifically, the correlation between $\mathbf{x}$ and $\mathbf{y}$ is identical to the cosine between $\mathbf{x}'$ and $\mathbf{y}'$. However, the cosine between $\mathbf{x}$ and $\mathbf{y}$ is not the same as the cosine between $\mathbf{x}'$ and $\mathbf{y}'$, even though they both have the same correlation measure. In general, the correlation between two vectors is equal to the cosine measure only in the special case when the means of the two vectors are 0.

Differences Among Measures For Continuous Attributes

In this section, we illustrate the difference among the three proximity measures for continuous attributes that we have just defined: cosine, correlation, and Minkowski distance. Specifically, we consider two types of data transformations that are commonly used, namely, scaling (multiplication) by a constant factor and translation (addition) by a constant value. A proximity measure is considered to be invariant to a data transformation if its value remains unchanged even after performing the transformation. Table 2.12 compares the

Table 2.12. Properties of cosine, correlation, and Minkowski distance measures.

Property	Cosine	Correlation	Minkowski Distance
Invariant to scaling (multiplication)	Yes	Yes	No
Invariant to translation (addition)	No	Yes	No

behavior of cosine, correlation, and Minkowski distance measures regarding their invariance to scaling and translation operations. It can be seen that while correlation is invariant to both scaling and translation, cosine is only invariant to scaling but not to translation. Minkowski distance measures, on the other hand, are sensitive to both scaling and translation and are thus invariant to neither.

Let us consider an example to demonstrate the significance of these differences among different proximity measures.

Example 2.21 (Comparing proximity measures)**.** Consider the following two vectors $\mathbf{x}$ and $\mathbf{y}$ with seven numeric attributes.

$\mathbf{x} = (1, 2, 4, 3, 0, 0, 0)$
$\mathbf{y} = (1, 2, 3, 4, 0, 0, 0)$

It can be seen that both $\mathbf{x}$ and $\mathbf{y}$ have 4 non-zero values, and the values in the two vectors are mostly the same, except for the third and the fourth components. The cosine, correlation, and Euclidean distance between the two vectors can be computed as follows.

$$\begin{aligned} \mathbf{cos}(\mathbf{x}, \mathbf{y}) &= \frac{29}{\sqrt{30} \times \sqrt{30}} &&= 0.9667 \\ \mathbf{correlation}(\mathbf{x}, \mathbf{y}) &= \frac{2.3571}{1.5811 \times 1.5811} &&= 0.9429 \\ \mathbf{Euclidean\ distance}(\mathbf{x}, \mathbf{y}) &= ||\mathbf{x} - \mathbf{y}|| &&= 1.4142 \end{aligned}$$

Not surprisingly, $\mathbf{x}$ and $\mathbf{y}$ have a cosine and correlation measure close to 1, while the Euclidean distance between them is small, indicating that they are quite similar. Now let us consider the vector $\mathbf{y_s}$, which is a scaled version of $\mathbf{y}$ (multiplied by a constant factor of 2), and the vector $\mathbf{y_t}$, which is constructed by translating $\mathbf{y}$ by 5 units as follows.

$\mathbf{y_s} = 2 \times \mathbf{y} = (2, 4, 6, 8, 0, 0, 0)$

Table 2.13. Similarity between $(\mathbf{x}, \mathbf{y})$, $(\mathbf{x}, \mathbf{y_s})$, and $(\mathbf{x}, \mathbf{y_t})$.

Measure	$(\mathbf{x}, \mathbf{y})$	$(\mathbf{x}, \mathbf{y_s})$	$(\mathbf{x}, \mathbf{y_t})$
Cosine	0.9667	0.9667	0.7940
Correlation	0.9429	0.9429	0.9429
Euclidean Distance	1.4142	5.8310	14.2127

$\mathbf{y_t} = \mathbf{y} + 5 = (6, 7, 8, 9, 5, 5, 5)$

We are interested in finding whether $\mathbf{y_s}$ and $\mathbf{y_t}$ show the same proximity with $\mathbf{x}$ as shown by the original vector $\mathbf{y}$. Table 2.13 shows the different measures of proximity computed for the pairs $(\mathbf{x}, \mathbf{y})$, $(\mathbf{x}, \mathbf{y_s})$, and $(\mathbf{x}, \mathbf{y_t})$. It can be seen that the value of correlation between $\mathbf{x}$ and $\mathbf{y}$ remains unchanged even after replacing $\mathbf{y}$ with $\mathbf{y_s}$ or $\mathbf{y_t}$. However, the value of cosine remains equal to 0.9667 when computed for $(\mathbf{x}, \mathbf{y})$ and $(\mathbf{x}, \mathbf{y_s})$, but significantly reduces to 0.7940 when computed for $(\mathbf{x}, \mathbf{y_t})$. This highlights the fact that cosine is invariant to the scaling operation but not to the translation operation, in contrast with the correlation measure. The Euclidean distance, on the other hand, shows different values for all three pairs of vectors, as it is sensitive to both scaling and translation.

We can observe from this example that different proximity measures behave differently when scaling or translation operations are applied on the data. The choice of the right proximity measure thus depends on the desired notion of similarity between data objects that is meaningful for a given application. For example, if $\mathbf{x}$ and $\mathbf{y}$ represented the frequencies of different words in a document-term matrix, it would be meaningful to use a proximity measure that remains unchanged when $\mathbf{y}$ is replaced by $\mathbf{y_s}$, because $\mathbf{y_s}$ is just a scaled version of $\mathbf{y}$ with the same distribution of words occurring in the document. However, $\mathbf{y_t}$ is different from $\mathbf{y}$, since it contains a large number of words with non-zero frequencies that do not occur in $\mathbf{y}$. Because cosine is invariant to scaling but not to translation, it will be an ideal choice of proximity measure for this application.

Consider a different scenario in which $\mathbf{x}$ represents a location's temperature measured on the Celsius scale for seven days. Let $\mathbf{y}$, $\mathbf{y_s}$, and $\mathbf{y_t}$ be the temperatures measured on those days at a different location, but using three different measurement scales. Note that different units of temperature have different offsets (e.g., Celsius and Kelvin) and different scaling factors (e.g., Celsius and Fahrenheit). It is thus desirable to use a proximity measure that captures the proximity between temperature values without being affected by

the measurement scale. Correlation would then be the ideal choice of proximity measure for this application, as it is invariant to both scaling and translation.

As another example, consider a scenario where $\mathbf{x}$ represents the amount of precipitation (in cm) measured at seven locations. Let $\mathbf{y}$, $\mathbf{y_s}$, and $\mathbf{y_t}$ be estimates of the precipitation at these locations, which are predicted using three different models. Ideally, we would like to choose a model that accurately reconstructs the measurements in $\mathbf{x}$ without making any error. It is evident that $\mathbf{y}$ provides a good approximation of the values in $\mathbf{x}$, whereas $\mathbf{y_s}$ and $\mathbf{y_t}$ provide poor estimates of precipitation, even though they do capture the trend in precipitation across locations. Hence, we need to choose a proximity measure that penalizes any difference in the model estimates from the actual observations, and is sensitive to both the scaling and translation operations. The Euclidean distance satisfies this property and thus would be the right choice of proximity measure for this application. Indeed, the Euclidean distance is commonly used in computing the accuracy of models, which will be discussed later in Chapter 3.

■

2.4.6 Mutual Information

Like correlation, mutual information is used as a measure of similarity between two sets of paired values that is sometimes used as an alternative to correlation, particularly when a nonlinear relationship is suspected between the pairs of values. This measure comes from information theory, which is the study of how to formally define and quantify information. Indeed, mutual information is a measure of how much information one set of values provides about another, given that the values come in pairs, e.g., height and weight. If the two sets of values are independent, i.e., the value of one tells us nothing about the other, then their mutual information is 0. On the other hand, if the two sets of values are completely dependent, i.e., knowing the value of one tells us the value of the other and vice-versa, then they have maximum mutual information. Mutual information does not have a maximum value, but we will define a normalized version of it that ranges between 0 and 1.

To define mutual information, we consider two sets of values, X and Y, which occur in pairs (X, Y). We need to measure the average information in a single set of values, i.e., either in X or in Y, and in the pairs of their values. This is commonly measured by entropy. More specifically, assume X and Y are discrete, that is, X can take m distinct values, $u_1, u_2, \ldots, u_m$ and Y can take n distinct values, $v_1, v_2, \ldots, v_n$. Then their individual and joint entropy

can be defined in terms of the probabilities of each value and pair of values as follows:

$$H(X) = -\sum_{j=1}^{m} P(X = u_j) \log_2 P(X = u_j) \tag{2.12}$$

$$H(Y) = -\sum_{k=1}^{n} P(Y = v_k) \log_2 P(Y = v_k) \tag{2.13}$$

$$H(X, Y) = -\sum_{j=1}^{m} \sum_{k=1}^{n} P(X = u_j, Y = v_k) \log_2 P(X = u_j, Y = v_k) \tag{2.14}$$

where if the probability of a value or combination of values is 0, then $0 \log_2(0)$ is conventionally taken to be 0.

The mutual information of X and Y can now be defined straightforwardly:

$$I(X, Y) = H(X) + H(Y) - H(X, Y) \tag{2.15}$$

Note that $H(X, Y)$ is symmetric, i.e., $H(X, Y) = H(Y, X)$, and thus mutual information is also symmetric, i.e., $I(X, Y) = I(Y)$.

Practically, X and Y are either the values in two attributes or two rows of the same data set. In Example 2.22, we will represent those values as two vectors $\mathbf{x}$ and $\mathbf{y}$ and calculate the probability of each value or pair of values from the frequency with which values or pairs of values occur in $\mathbf{x}$, $\mathbf{y}$ and (x_i, y_i), where x_i is the ith component of $\mathbf{x}$ and y_i is the ith component of $\mathbf{y}$. Let us illustrate using a previous example.

Example 2.22 (Evaluating Nonlinear Relationships with Mutual Information)**.** Recall Example 2.19 where $y_k = x_k^2$, but their correlation was 0.

$\mathbf{x} = (-3, -2, -1,\ 0,\ 1,\ 2,\ 3)$
$\mathbf{y} = (\ 9,\ \ 4,\ \ 1,\ 0,\ 1,\ 4,\ 9)$

From Figure 2.22, $I(\mathbf{x}, \mathbf{y}) = H(\mathbf{x}) + H(\mathbf{y}) - H(\mathbf{x}, \mathbf{y}) = 1.9502$. Although a variety of approaches to normalize mutual information are possible—see Bibliographic Notes—for this example, we will apply one that divides the mutual information by $\log_2(\min(m, n))$ and produces a result between 0 and 1. This yields a value of $1.9502/\log_2(4)) = 0.9751$. Thus, we can see that $\mathbf{x}$ and $\mathbf{y}$ are strongly related. They are not perfectly related because given a value of $\mathbf{y}$ there is, except for $\mathbf{y} = 0$, some ambiguity about the value of $\mathbf{x}$. Notice that for $\mathbf{y} = -\mathbf{x}$, the normalized mutual information would be 1. ■

Figure 2.18. Computation of mutual information.

Table 2.14. Entropy for $\mathbf{x}$

x_j	$P(\mathbf{x} = x_j)$	$-P(\mathbf{x} = x_j) \log_2 P(\mathbf{x} = x_j)$
-3	1/7	0.4011
-2	1/7	0.4011
-1	1/7	0.4011
0	1/7	0.4011
1	1/7	0.4011
2	1/7	0.4011
3	1/7	0.4011
$H(\mathbf{x})$		2.8074

Table 2.15. Entropy for $\mathbf{y}$

y_k	$P(\mathbf{y} = y_k)$	$-P(\mathbf{y} = y_k) \log_2(P(\mathbf{y} = y_k)$
9	2/7	0.5164
4	2/7	0.5164
1	2/7	0.5164
0	1/7	0.4011
$H(\mathbf{y})$		1.9502

Table 2.16. Joint entropy for $\mathbf{x}$ and $\mathbf{y}$

x_j	y_k	$P(\mathbf{x} = x_j, \mathbf{y} = x_k)$	$-P(\mathbf{x} = x_j, \mathbf{y} = x_k) \log_2 P(\mathbf{x} = x_j, \mathbf{y} = x_k)$
-3	9	1/7	0.4011
-2	4	1/7	0.4011
-1	1	1/7	0.4011
0	0	1/7	0.4011
1	1	1/7	0.4011
2	4	1/7	0.4011
3	9	1/7	0.4011
$H(\mathbf{x}, \mathbf{y})$			2.8074

2.4.7 Kernel Functions*

It is easy to understand how similarity and distance might be useful in an application such as clustering, which tries to group similar objects together. What is much less obvious is that many other data analysis tasks, including predictive modeling and dimensionality reduction, can be expressed in terms of pairwise "proximities" of data objects. More specifically, many data analysis problems can be mathematically formulated to take as input, a **kernel matrix**, $\mathbf{K}$, which can be considered a type of proximity matrix. Thus, an initial preprocessing step is used to convert the input data into a kernel matrix, which is the input to the data analysis algorithm.

More formally, if a data set has m data objects, then $\mathbf{K}$ is an m by m matrix. If $\mathbf{x}_i$ and $\mathbf{x}_j$ are the ith and j^{th} data objects, respectively, then k_{ij}, the ij^{th} entry of $\mathbf{K}$, is computed by a **kernel function**:

$$k_{ij} = \kappa(\mathbf{x}_i, \mathbf{x}_j) \tag{2.16}$$

As we will see in the material that follows, the use of a kernel matrix allows both wider applicability of an algorithm to various kinds of data and an ability to model nonlinear relationships with algorithms that are designed only for detecting linear relationships.

Kernels make an algorithm data independent If an algorithm uses a kernel matrix, then it can be used with any type of data for which a kernel function can be designed. This is illustrated by Algorithm 2.1. Although only some data analysis algorithms can be modified to use a kernel matrix as input, this approach is extremely powerful because it allows such an algorithm to be used with almost any type of data for which an appropriate kernel function can be defined. Thus, a classification algorithm can be used, for example, with record data, string data, or graph data. If an algorithm can be reformulated to use a kernel matrix, then its applicability to different types of data increases dramatically. As we will see in later chapters, many clustering, classification, and anomaly detection algorithms work only with similarities or distances, and thus, can be easily modified to work with kernels.

Algorithm 2.1 Basic kernel algorithm.

1: Read in the m data objects in the data set.
2: Compute the kernel matrix, $\mathbf{K}$ by applying the kernel function, κ, to each pair of data objects.
3: Run the data analysis algorithm with $\mathbf{K}$ as input.
4: Return the analysis result, e.g., predicted class or cluster labels.

Mapping data into a higher dimensional data space can allow modeling of nonlinear relationships There is yet another, equally important, aspect of kernel based data algorithms—their ability to model nonlinear relationships with algorithms that model only linear relationships. Typically, this works by first transforming (mapping) the data from a lower dimensional data space to a higher dimensional space.

Example 2.23 (Mapping Data to a Higher Dimensional Space)**.** Consider the relationship between two variables x and y given by the following equation, which defines an ellipse in two dimensions (Figure 2.19(a)):

$$4x^2 + 9xy + 7y^2 = 10 \tag{2.17}$$

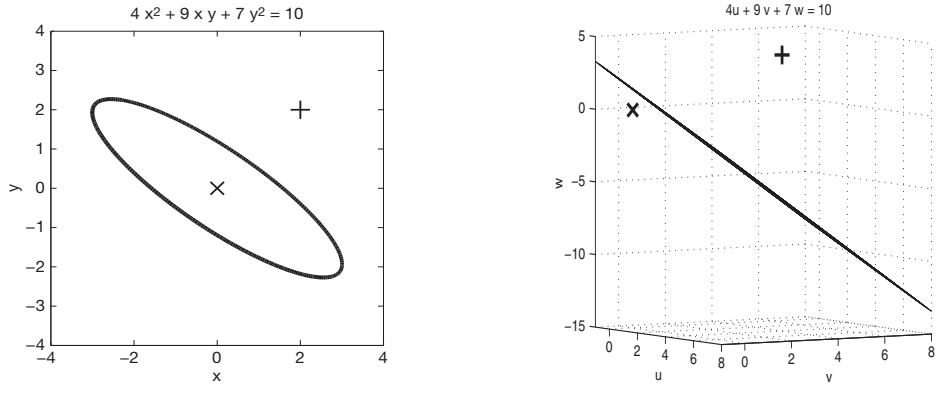
We can map our two dimensional data to three dimensions by creating three new variables, u, v, and w, which are defined as follows:

$$\begin{aligned} w &= x^2 \\ u &= xy \\ v &= y^2 \end{aligned}$$

As a result, we can now express Equation 2.17 as a linear one. This equation describes a plane in three dimensions. Points on the ellipse will lie on that plane, while points inside and outside the ellipse will lie on opposite sides of the plane. See Figure 2.19(b). The viewpoint of this 3D plot is along the surface of the separating plane so that the plane appears as a line.

$$4u + 9v + 7w = 10 \tag{2.18}$$

■



(a) Ellipse and two points in 2 dimensions.

(b) Data mapped to 3 dimensions.

Figure 2.19. Mapping data to a higher dimensional space: two to three dimensions.

The Kernel Trick The approach illustrated above shows the value in mapping data to higher dimensional space, an operation that is integral to kernel-based methods. Conceptually, we first define a function φ that maps data points $\mathbf{x}$ and $\mathbf{y}$ to data points $\varphi(\mathbf{x})$ and $\varphi(\mathbf{y})$ in a higher dimensional space

such that the inner product $\langle \mathbf{x}, \mathbf{y} \rangle$ gives the desired measure of proximity of $\mathbf{x}$ and $\mathbf{y}$. It may seem that we have potentially sacrificed a great deal by using such an approach, because we can greatly expand the size of our data, increase the computational complexity of our analysis, and encounter problems with the curse of dimensionality by computing similarity in a high-dimensional space. However, this is not the case since these problems can be avoided by defining a kernel function κ that can compute the same similarity value, but with the data points in the original space, i.e., $\kappa(\mathbf{x}, \mathbf{y}) = \langle \varphi(\mathbf{x}), \varphi(\mathbf{y}) \rangle$. This is known as the *kernel trick*. Despite the name, the kernel trick has a very solid mathematical foundation and is a remarkably powerful approach for data analysis.

Not every function of a pair of data objects satisfies the properties needed for a kernel function, but it has been possible to design many useful kernels for a wide variety of data types. For example, three common kernel functions are the polynomial, Gaussian (radial basis function (RBF)), and sigmoid kernels. If $\mathbf{x}$ and $\mathbf{y}$ are two data objects, specifically, two data vectors, then these two kernel functions can be expressed as follows, respectively:

$$\kappa(\mathbf{x}, \mathbf{y}) = (\mathbf{x}'\mathbf{y} + c)^d \tag{2.19}$$

$$\kappa(\mathbf{x}, \mathbf{y}) = \exp(-||\mathbf{x} - \mathbf{y}||/2\sigma^2) \tag{2.20}$$

$$\kappa(\mathbf{x}, \mathbf{y}) = \tanh(\alpha \mathbf{x}'\mathbf{y} + c) \tag{2.21}$$

where α and $c \geq 0$ are constants, d is an integer parameter that gives the polynomial degree, $||\mathbf{x} - \mathbf{y}||$ is the length of the vector $\mathbf{x} - \mathbf{y}$ and $\sigma > 0$ is a parameter that governs the "spread" of a Gaussian.

Example 2.24 (The Polynomial Kernel)**.** Note that the kernel functions presented in the previous section are computing the same similarity value as would be computed if we actually mapped the data to a higher dimensional space and then computed an inner product there. For example, for the polynomial kernel of degree 2, let φ be the function that maps a two-dimensional data vector $\mathbf{x} = (x_1, x_2)$ to the higher dimensional space. Specifically, let

$$\varphi(\mathbf{x}) = (x_1^2,\ x_2^2,\ \sqrt{2}x_1x_2,\ \sqrt{2c}x_1, \sqrt{2c}x_2,\ c). \tag{2.22}$$

For the higher dimensional space, let the proximity be defined as the inner product of $\varphi(\mathbf{x})$ and $\varphi(\mathbf{y})$, i.e., $\langle \varphi(\mathbf{x}), \varphi(\mathbf{y}) \rangle$. Then, as previously mentioned, it can be shown that

$$\kappa(\mathbf{x}, \mathbf{y}) = \langle \varphi(\mathbf{x}), \varphi(\mathbf{y}) \rangle \tag{2.23}$$

where κ is defined by Equation 2.19 above. Specifically, if $\mathbf{x} = (x_1, x_2)$ and $\mathbf{y} = (y_1, y_2)$, then

$$\kappa(\mathbf{x}, \mathbf{y}) = \langle \mathbf{x}, \mathbf{y} \rangle = \mathbf{x}'\mathbf{y} = (x_1^2 y_1^2,\ x_2^2 y_2^2,\ 2x_1 x_2 y_1 y_2,\ 2cx_1 y_1, 2cx_2 y_2,\ c^2). \quad (2.24)$$

More generally, the kernel trick depends on defining κ and φ so that Equation 2.23 holds. This has been done for a wide variety of kernels. ■

This discussion of kernel-based approaches was intended only to provide a brief introduction to this topic and has omitted many details. A fuller discussion of the kernel-based approach is provided in Section 6.9.4, which discusses these issues in the context of nonlinear support vector machines for classification. More general references for the kernel based analysis can be found in the Bibliographic Notes of this chapter.

2.4.8 Bregman Divergence*

This section provides a brief description of Bregman divergences, which are a family of proximity functions that share some common properties. As a result, it is possible to construct general data mining algorithms, such as clustering algorithms, that work with any Bregman divergence. A concrete example is the K-means clustering algorithm (Section 5.2). Note that this section requires knowledge of vector calculus.

Bregman divergences are loss or distortion functions. To understand the idea of a loss function, consider the following. Let $\mathbf{x}$ and $\mathbf{y}$ be two points, where $\mathbf{y}$ is regarded as the original point and $\mathbf{x}$ is some distortion or approximation of it. For example, $\mathbf{x}$ may be a point that was generated by adding random noise to $\mathbf{y}$. The goal is to measure the resulting distortion or loss that results if $\mathbf{y}$ is approximated by $\mathbf{x}$. Of course, the more similar $\mathbf{x}$ and $\mathbf{y}$ are, the smaller the loss or distortion. Thus, Bregman divergences can be used as dissimilarity functions.

More formally, we have the following definition.

Definition 2.6 (Bregman divergence)**.** Given a strictly convex function ϕ (with a few modest restrictions that are generally satisfied), the Bregman divergence (loss function) $D(\mathbf{x}, \mathbf{y})$ generated by that function is given by the following equation:

$$D(\mathbf{x}, \mathbf{y}) = \phi(\mathbf{x}) - \phi(\mathbf{y}) - \langle \nabla\phi(\mathbf{y}), (\mathbf{x} - \mathbf{y}) \rangle \quad (2.25)$$

where $\nabla\phi(\mathbf{y})$ is the gradient of ϕ evaluated at $\mathbf{y}$, $\mathbf{x} - \mathbf{y}$, is the vector difference between $\mathbf{x}$ and $\mathbf{y}$, and $\langle \nabla\phi(\mathbf{y}), (\mathbf{x} - \mathbf{y}) \rangle$ is the inner product between $\nabla\phi(\mathbf{y})$

and $(\mathbf{x} - \mathbf{y})$. For points in Euclidean space, the inner product is just the dot product.

$D(\mathbf{x}, \mathbf{y})$ can be written as $D(\mathbf{x}, \mathbf{y}) = \phi(\mathbf{x}) - L(\mathbf{x})$, where $L(\mathbf{x}) = \phi(\mathbf{y}) + \langle \nabla\phi(\mathbf{y}), (\mathbf{x} - \mathbf{y}) \rangle$ and represents the equation of a plane that is tangent to the function ϕ at $\mathbf{y}$. Using calculus terminology, $L(\mathbf{x})$ is the linearization of ϕ around the point $\mathbf{y}$, and the Bregman divergence is just the difference between a function and a linear approximation to that function. Different Bregman divergences are obtained by using different choices for ϕ.

Example 2.25. We provide a concrete example using squared Euclidean distance, but restrict ourselves to one dimension to simplify the mathematics. Let x and y be real numbers and $\phi(t)$ be the real-valued function, $\phi(t) = t^2$. In that case, the gradient reduces to the derivative, and the dot product reduces to multiplication. Specifically, Equation 2.25 becomes Equation 2.26.

$$D(x, y) = x^2 - y^2 - 2y(x - y) = (x - y)^2 \tag{2.26}$$

The graph for this example, with $y = 1$, is shown in Figure 2.20. The Bregman divergence is shown for two values of x: $x = 2$ and $x = 3$. ■

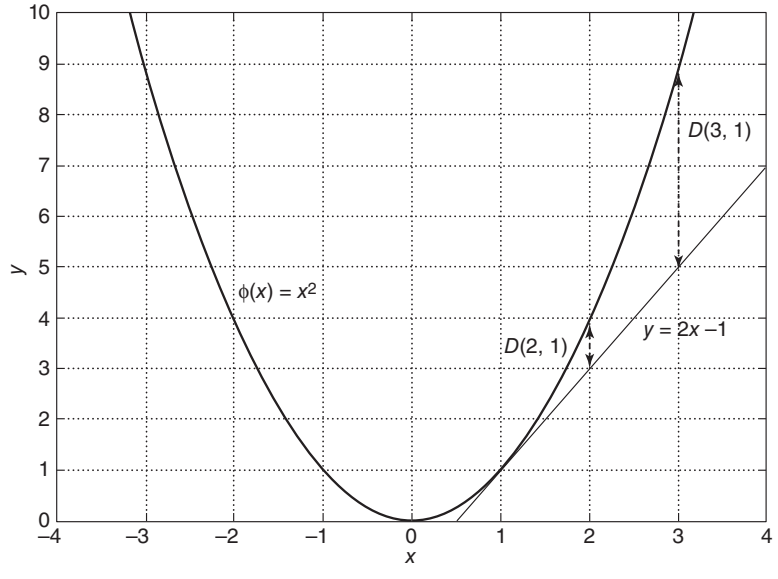


Figure 2.20. Illustration of Bregman divergence.

2.4.9 Issues in Proximity Calculation

This section discusses several important issues related to proximity measures: (1) how to handle the case in which attributes have different scales and/or are correlated, (2) how to calculate proximity between objects that are composed of different types of attributes, e.g., quantitative and qualitative, (3) and how to handle proximity calculations when attributes have different weights; i.e., when not all attributes contribute equally to the proximity of objects.

Standardization and Correlation for Distance Measures

An important issue with distance measures is how to handle the situation when attributes do not have the same range of values. (This situation is often described by saying that "the variables have different scales.") In a previous example, Euclidean distance was used to measure the distance between people based on two attributes: age and income. Unless these two attributes are standardized, the distance between two people will be dominated by income.

A related issue is how to compute distance when there is correlation between some of the attributes, perhaps in addition to differences in the ranges of values. A generalization of Euclidean distance, the **Mahalanobis distance**, is useful when attributes are correlated, have different ranges of values (different variances), and the distribution of the data is approximately Gaussian (normal). Correlated variables have a large impact on standard distance measures since a change in any of the correlated variables is reflected in a change in all the correlated variables. Specifically, the Mahalanobis distance between two objects (vectors) $\mathbf{x}$ and $\mathbf{y}$ is defined as

$$\text{Mahalanobis}(\mathbf{x}, \mathbf{y}) = \sqrt{(\mathbf{x} - \mathbf{y})' \boldsymbol{\Sigma}^{-1} (\mathbf{x} - \mathbf{y})}, \tag{2.27}$$

where $\boldsymbol{\Sigma}^{-1}$ is the inverse of the covariance matrix of the data. Note that the covariance matrix $\boldsymbol{\Sigma}$ is the matrix whose ij^{th} entry is the covariance of the ith and j^{th} attributes as defined by Equation 2.11.

Example 2.26. In Figure 2.21, there are 1000 points, whose x and y attributes have a correlation of 0.6. The distance between the two large points at the opposite ends of the long axis of the ellipse is 14.7 in terms of Euclidean distance, but only 6 with respect to Mahalanobis distance. This is because the Mahalanobis distance gives less emphasis to the direction of largest variance. In practice, computing the Mahalanobis distance is expensive, but can be worthwhile for data whose attributes are correlated. If the attributes are relatively uncorrelated, but have different ranges, then standardizing the variables is sufficient.

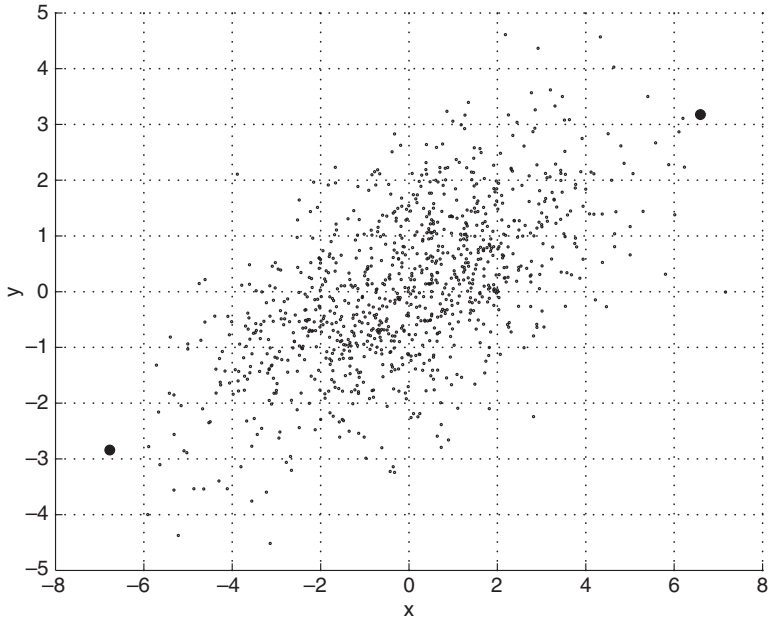


Figure 2.21. Set of two-dimensional points. The Mahalanobis distance between the two points represented by large dots is 6; their Euclidean distance is 14.7.

■

Combining Similarities for Heterogeneous Attributes

The previous definitions of similarity were based on approaches that assumed all the attributes were of the same type. A general approach is needed when the attributes are of different types. One straightforward approach is to compute the similarity between each attribute separately using Table 2.7, and then combine these similarities using a method that results in a similarity between 0 and 1. One possible approach is to define the overall similarity as the average of all the individual attribute similarities. Unfortunately, this approach does not work well if some of the attributes are asymmetric attributes. For example, if all the attributes are asymmetric binary attributes, then the similarity measure suggested previously reduces to the simple matching coefficient, a measure that is not appropriate for asymmetric binary attributes. The easiest way to fix this problem is to omit asymmetric attributes from the similarity calculation when their values are 0 for both of the objects whose similarity

is being computed. A similar approach also works well for handling missing values.

In summary, Algorithm 2.2 is effective for computing an overall similarity between two objects, $\mathbf{x}$ and $\mathbf{y}$, with different types of attributes. This procedure can be easily modified to work with dissimilarities.

Algorithm 2.2 Similarities of heterogeneous objects.

1: For the kth attribute, compute a similarity, $s_k(\mathbf{x}, \mathbf{y})$, in the range $[0, 1]$.
2: Define an indicator variable, δ_k, for the kth attribute as follows:

$$\delta_k = \begin{cases} 0 & \text{if the } k^{th} \text{ attribute is an asymmetric attribute and} \\ & \text{both objects have a value of 0, or if one of the objects} \\ & \text{has a missing value for the } k^{th} \text{ attribute} \\ 1 & \text{otherwise} \end{cases}$$

3: Compute the overall similarity between the two objects using the following formula:

$$\text{similarity}(\mathbf{x}, \mathbf{y}) = \frac{\sum_{k=1}^{n} \delta_k s_k(\mathbf{x}, \mathbf{y})}{\sum_{k=1}^{n} \delta_k} \tag{2.28}$$

Using Weights

In much of the previous discussion, all attributes were treated equally when computing proximity. This is not desirable when some attributes are more important to the definition of proximity than others. To address these situations, the formulas for proximity can be modified by weighting the contribution of each attribute.

With attribute weights, w_k, (2.28) becomes

$$\text{similarity}(\mathbf{x}, \mathbf{y}) = \frac{\sum_{k=1}^{n} w_k \delta_k s_k(\mathbf{x}, \mathbf{y})}{\sum_{k=1}^{n} w_k \delta_k}. \tag{2.29}$$

The definition of the Minkowski distance can also be modified as follows:

$$d(\mathbf{x}, \mathbf{y}) = \left(\sum_{k=1}^{n} w_k |x_k - y_k|^r \right)^{1/r}. \tag{2.30}$$

2.4.10 Selecting the Right Proximity Measure

A few general observations may be helpful. First, the type of proximity measure should fit the type of data. For many types of dense, continuous data,

metric distance measures such as Euclidean distance are often used. Proximity between continuous attributes is most often expressed in terms of differences, and distance measures provide a well-defined way of combining these differences into an overall proximity measure. Although attributes can have different scales and be of differing importance, these issues can often be dealt with as described earlier, such as normalization and weighting of attributes.

For sparse data, which often consists of asymmetric attributes, we typically employ similarity measures that ignore 0–0 matches. Conceptually, this reflects the fact that, for a pair of complex objects, similarity depends on the number of characteristics they both share, rather than the number of characteristics they both lack. The cosine, Jaccard, and extended Jaccard measures are appropriate for such data.

There are other characteristics of data vectors that often need to be considered. Invariance to scaling (multiplication) and to translation (addition) were previously discussed with respect to Euclidean distance and the cosine and correlation measures. The practical implications of such considerations are that, for example, cosine is more suitable for sparse document data where only scaling is important, while correlation works better for time series, where both scaling and translation are important. Euclidean distance or other types of Minkowski distance are most appropriate when two data vectors are to match as closely as possible across all components (features).

In some cases, transformation or normalization of the data is needed to obtain a proper similarity measure. For instance, time series can have trends or periodic patterns that significantly impact similarity. Also, a proper computation of similarity often requires that time lags be taken into account. Finally, two time series may be similar only over specific periods of time. For example, there is a strong relationship between temperature and the use of natural gas, but only during the heating season.

Practical consideration can also be important. Sometimes, one or more proximity measures are already in use in a particular field, and thus, others will have answered the question of which proximity measures should be used. Other times, the software package or clustering algorithm being used can drastically limit the choices. If efficiency is a concern, then we may want to choose a proximity measure that has a property, such as the triangle inequality, that can be used to reduce the number of proximity calculations. (See Exercise 30.)

However, if common practice or practical restrictions do not dictate a choice, then the proper choice of a proximity measure can be a time-consuming task that requires careful consideration of both domain knowledge and the purpose for which the measure is being used. A number of different similarity

Table 6.5. Comparing the accuracy of a decision tree classifier against three ensemble methods.

Data Set	Number of (Attributes, Classes, Instances)	Decision Tree (%)	Bagging (%)	Boosting (%)	RF (%)
Anneal	(39, 6, 898)	92.09	94.43	95.43	95.43
Australia	(15, 2, 690)	85.51	87.10	85.22	85.80
Auto	(26, 7, 205)	81.95	85.37	85.37	84.39
Breast	(11, 2, 699)	95.14	96.42	97.28	96.14
Cleve	(14, 2, 303)	76.24	81.52	82.18	82.18
Credit	(16, 2, 690)	85.8	86.23	86.09	85.8
Diabetes	(9, 2, 768)	72.40	76.30	73.18	75.13
German	(21, 2, 1000)	70.90	73.40	73.00	74.5
Glass	(10, 7, 214)	67.29	76.17	77.57	78.04
Heart	(14, 2, 270)	80.00	81.48	80.74	83.33
Hepatitis	(20, 2, 155)	81.94	81.29	83.87	83.23
Horse	(23, 2, 368)	85.33	85.87	81.25	85.33
Ionosphere	(35, 2, 351)	89.17	92.02	93.73	93.45
Iris	(5, 3, 150)	94.67	94.67	94.00	93.33
Labor	(17, 2, 57)	78.95	84.21	89.47	84.21
Led7	(8, 10, 3200)	73.34	73.66	73.34	73.06
Lymphography	(19, 4, 148)	77.03	79.05	85.14	82.43
Pima	(9, 2, 768)	74.35	76.69	73.44	77.60
Sonar	(61, 2, 208)	78.85	78.85	84.62	85.58
Tic-tac-toe	(10, 2, 958)	83.72	93.84	98.54	95.82
Vehicle	(19, 4, 846)	71.04	74.11	78.25	74.94
Waveform	(22, 3, 5000)	76.44	83.30	83.90	84.04
Wine	(14, 3, 178)	94.38	96.07	97.75	97.75
Zoo	(17, 7, 101)	93.07	93.07	95.05	97.03

trees. The classification accuracies reported in this table are obtained from ten-fold cross-validation. Notice that the ensemble classifiers generally outperform a single decision tree classifier on many of the data sets.

6.11 Class Imbalance Problem

In many data sets there are a disproportionate number of instances that belong to different classes, a property known as **skew** or **class imbalance**. For example, consider a health-care application where diagnostic reports are used to decide whether a person has a rare disease. Because of the infrequent nature of the disease, we can expect to observe a smaller number of subjects who are positively diagnosed. Similarly, in credit card fraud detection, fraudulent transactions are greatly outnumbered by legitimate transactions.

The degree of imbalance between the classes varies across different applications and even across different data sets from the same application. For example, the risk for a rare disease may vary across different populations of subjects depending on their dietary and lifestyle choices. However, despite their infrequent occurrences, a correct classification of the rare class often has greater value than a correct classification of the majority class. For example, it may be more dangerous to ignore a patient suffering from a disease than to misdiagnose a healthy person.

More generally, class imbalance poses two challenges for classification. First, it can be difficult to find sufficiently many labeled samples of a rare class. Note that many of the classification methods discussed so far work well only when the training set has a balanced representation of both classes. Although some classifiers are more effective at handling imbalance in the training data than others, e.g., rule-based classifiers and k-NN, they are all impacted if the minority class is not well-represented in the training set. In general, a classifier trained over an imbalanced data set shows a bias toward improving its performance over the majority class, which is often not the desired behavior. As a result, many existing classification models, when trained on an imbalanced data set, may not effectively detect instances of the rare class.

Second, accuracy, which is the traditional measure for evaluating classification performance, is not well-suited for evaluating models in the presence of class imbalance in the test data. For example, if 1% of the credit card transactions are fraudulent, then a trivial model that predicts every transaction as legitimate will have an accuracy of 99% even though it fails to detect any of the fraudulent activities. Thus, there is a need to use alternative evaluation metrics that are sensitive to the skew and can capture different criteria of performance than accuracy.

In this section, we first present some of the generic methods for building classifiers when there is class imbalance in the training set. We then discuss methods for evaluating classification performance and adapting classification decisions in the presence of a skewed test set. In the remainder of this section, we will consider binary classification problems for simplicity, where the minority class is referred as the positive (+) class while the majority class is referred as the negative (−) class.

6.11.1 Building Classifiers with Class Imbalance

There are two primary considerations for building classifiers in the presence of class imbalance in the training set. First, we need to ensure that the learning

algorithm is trained over a data set that has adequate representation of both the majority as well as the minority classes. Some common approaches for ensuring this includes the methodologies of oversampling and undersampling the training set. Second, having learned a classification model, we need a way to adapt its classification decisions (and thus create an appropriately tuned classifier) to best match the requirements of the imbalanced test set. This is typically done by converting the outputs of the classification model to real-valued scores, and then selecting a suitable threshold on the classification score to match the needs of a test set. Both these considerations are discussed in detail in the following.

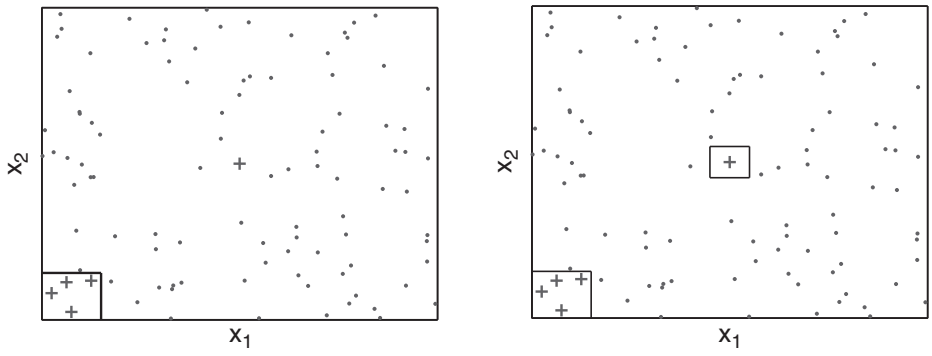
Oversampling and Undersampling

The first step in learning with imbalanced data is to transform the training set to a balanced training set, where both classes have nearly equal representation. The balanced training set can then be used with any of the existing classification techniques (without making any modifications in the learning algorithm) to learn a model that gives equal emphasis to both classes. In the following, we present some of the common techniques for transforming an imbalanced training set to a balanced one.

A basic approach for creating balanced training sets is to generate a *sample* of training instances where the rare class has adequate representation. There are two types of sampling methods that can be used to enhance the representation of the minority class: (a) **undersampling**, where the frequency of the majority class is reduced to match the frequency of the minority class, and (b) **oversampling**, where artificial examples of the minority class are created to make them equal in proportion to the number of negative instances.

To illustrate undersampling, consider a training set that contains 100 positive examples and 1000 negative examples. To overcome the skew among the classes, we can select a random sample of 100 examples from the negative class and use them with the 100 positive examples to create a balanced training set. A classifier built over the resultant balanced set will then be unbiased toward both classes. However, one limitation of undersampling is that some of the useful negative examples (e.g., those closer to the actual decision boundary) may not be chosen for training, therefore, resulting in an inferior classification model. Another limitation is that the smaller sample of 100 negative instances may have a higher variance than the larger set of 1000.

Oversampling attempts to create a balanced training set by artificially generating new positive examples. A simple approach for oversampling is to duplicate every positive instance n_-/n_+ times, where n_+ and n_- are the



(a) Without oversampling. (b) With oversampling.

Figure 6.51. Illustrating the effect of oversampling of the rare class.

numbers of positive and negative training instances, respectively. Figure 6.51 illustrates the effect of oversampling on the learning of a decision boundary using a classifier such as a decision tree. Without oversampling, only the positive examples at the bottom right-hand side of Figure 6.51(a) are classified correctly. The positive example in the middle of the diagram is misclassified because there are not enough examples to justify the creation of a new decision boundary to separate the positive and negative instances. Oversampling provides the additional examples needed to ensure that the decision boundary surrounding the positive example is not pruned, as illustrated in Figure 6.51(b). Note that duplicating a positive instance is analogous to doubling its weight during the training stage. Hence, the effect of oversampling can be alternatively achieved by assigning higher weights to positive instances than negative instances. This method of weighting instances can be used with a number of classifiers such as logistic regression, ANN, and SVM.

One limitation of the duplication method for oversampling is that the replicated positive examples have an artificially lower variance when compared with their true distribution in the overall data. This can bias the classifier to the specific distribution of training instances, which may not be representative of the overall distribution of test instances, leading to poor generalizability. To overcome this limitation, an alternative approach for oversampling is to generate *synthetic* positive instances in the neighborhood of existing positive instances. In this approach, called the Synthetic Minority Oversampling Technique (SMOTE), we first determine the k-nearest positive neighbors of every positive instance $\mathbf{x}$, and then generate a synthetic positive instance

at some intermediate point along the line segment joining $\mathbf{x}$ to one of its randomly chosen k-nearest neighbor, $\mathbf{x}_k$. This process is repeated until the desired number of positive instances is reached. However, one limitation of this approach is that it can only generate new positive instances in the convex hull of the existing positive class. Hence, it does not help improve the representation of the positive class outside the boundary of existing positive instances. Despite their complementary strengths and weaknesses, undersampling and oversampling provide useful directions for generating balanced training sets in the presence of class imbalance.

Assigning Scores to Test Instances

If a classifier returns an ordinal score $s(\mathbf{x})$ for every test instance $\mathbf{x}$ such that a higher score denotes a greater likelihood of $\mathbf{x}$ belonging to the positive class, then for every possible value of score threshold, s_T, we can create a new binary classifier where a test instance $\mathbf{x}$ is classified positive only if $s(\mathbf{x}) > s_T$. Thus, every choice of s_T can potentially lead to a different classifier, and we are interested in finding the classifier that is best suited for our needs.

Ideally, we would like the classification score to vary monotonically with the actual posterior probability of the positive class, i.e., if $s(\mathbf{x}_1)$ and $s(\mathbf{x}_2)$ are the scores of any two instances, $\mathbf{x}_1$ and $\mathbf{x}_2$, then $s(x_1) \geq s(x_2) \implies P(y = 1|x_1) \geq P(y|x_2)$. However, this is difficult to guarantee in practice as the properties of the classification score depends on several factors such as the complexity of the classification algorithm and the representative power of the training set. In general, we can only expect the classification score of a reasonable algorithm to be weakly related to the actual posterior probability of the positive class, even though the relationship may not be strictly monotonic. Most classifiers can be easily modified to produce such a real valued score. For example, the signed distance of an instance from the positive margin hyperplane of SVM can be used as a classification score. As another example, test instances belonging to a leaf in a decision tree can be assigned a score based on the fraction of training instances labeled as positive in the leaf. Also, probabilistic classifiers such as naïve Bayes, Bayesian networks, and logistic regression naturally output estimates of posterior probabilities, $P(y = 1|\mathbf{x})$. Next, we discuss some evaluation measures for assessing the goodness of a classifier in the presence of class imbalance.

Table 6.6. A confusion matrix for a binary classification problem in which the classes are not equally important.

		Predicted Class	
		+	−
Actual	+	f_{++} (TP)	f_{+-} (FN)
Class	−	f_{-+} (FP)	f_{--} (TN)

6.11.2 Evaluating Performance with Class Imbalance

The most basic approach for representing a classifier's performance on a test set is to use a **confusion matrix**, as shown in Table 6.6. This table is essentially the same as Table 3.4, which was introduced in the context of evaluating classification performance in Section 3.2. A confusion matrix summarizes the number of instances predicted correctly or incorrectly by a classifier using the following four counts:

- True positive (TP) or f_{++}, which corresponds to the number of positive examples correctly predicted by the classifier.
- False positive (FP) or f_{-+} (also known as Type I error), which corresponds to the number of negative examples wrongly predicted as positive by the classifier.
- False negative (FN) or f_{+-} (also known as Type II error), which corresponds to the number of positive examples wrongly predicted as negative by the classifier.
- True negative (TN) or f_{--}, which corresponds to the number of negative examples correctly predicted by the classifier.

The confusion matrix provides a concise representation of classification performance on a given test data set. However, it is often difficult to interpret and compare the performance of classifiers using the four-dimensional representations (corresponding to the four counts) provided by their confusion matrices. Hence, the counts in the confusion matrix are often summarized using a number of **evaluation measures**. Accuracy is an example of one such measure that combines these four counts into a single value, which is used extensively when classes are balanced. However, the accuracy measure is not suitable for handling data sets with imbalanced class distributions as it tends to favor classifiers that correctly classify the majority class. In the

following, we describe other possible measures that capture different criteria of performance when working with imbalanced classes.

A basic evaluation measure is the **true positive rate** (TPR), which is defined as the fraction of positive test instances correctly predicted by the classifier:

$$\text{TPR} = \frac{\text{TP}}{\text{TP} + \text{FN}}.$$

In the medical community, TPR is also known as **sensitivity**, while in the information retrieval literature, it is also called **recall** (r). A classifier with a high TPR has a high chance of correctly identifying the positive instances of the data.

Analogously to TPR, the **true negative rate** (TNR) (also known as **specificity**) is defined as the fraction of negative test instances correctly predicted by the classifier, i.e.,

$$\text{TNR} = \frac{\text{TN}}{\text{FP} + \text{TN}}.$$

A high TNR value signifies that the classifier correctly classifies any randomly chosen negative instance in the test set. A commonly used evaluation measure that is closely related to TNR is the **false positive rate** (FPR), which is defined as $1 - \text{TNR}$.

$$\text{FPR} = \frac{\text{FP}}{\text{FP} + \text{TN}}.$$

Similarly, we can define **false negative rate** (FNR) as $1 - \text{TPR}$.

$$\text{FNR} = \frac{\text{FN}}{\text{FN} + \text{TP}}.$$

Note that the evaluation measures defined above do not take into account the **skew** among the classes, which can be formally defined as $\alpha = P/(P+N)$, where P and N denote the number of actual positives and actual negatives, respectively. As a result, changing the relative numbers of P and N will have no effect on TPR, TNR, FPR, or FNR, since they depend only on the fraction of correct classifications for every class, independently of the other class. Furthermore, knowing the values of TPR and TNR (and consequently FNR and FPR) does not by itself help us uniquely determine all four entries of the confusion matrix. However, together with information about the skew factor, α, and the total number of instances, N, we can compute the entire confusion matrix using TPR and TNR, as shown in Table 6.7.

Table 6.7. Entries of the confusion matrix in terms of the TPR, TNR, skew, α, and total number of instances, N.

	Predicted +	Predicted −	
Actual +	TPR $\times$ $\alpha \times N$	(1 - TPR) $\times$ $\alpha \times N$	$\alpha \times N$
Actual −	(1 - TNR) $\times$ $(1-\alpha) \times N$	TNR $\times$ $(1-\alpha) \times N$	$(1-\alpha) \times N$
			N

An evaluation measure that is sensitive to the skew is **precision**, which can be defined as the fraction of correct predictions of the positive class over the total number of positive predictions, i.e.,

$$\text{Precision, } p = \frac{\text{TP}}{\text{TP} + \text{FP}}.$$

Precision is also referred as the **positive predicted value** (PPV). A classifier that has a high precision is likely to have most of its positive predictions correct. Precision is a useful measure for highly skewed test sets where the positive predictions, even though small in numbers, are required to be mostly correct. A measure that is closely related to precision is the **false discovery rate** (FDR), which can be defined as $1 - p$.

$$\text{FDR} = \frac{\text{FP}}{\text{TP} + \text{FP}}.$$

Although both FDR and FPR focus on FP, they are designed to capture different evaluation objectives and thus can take quite contrasting values, especially in the presence of class imbalance. To illustrate this, consider a classifier with the following confusion matrix.

		Predicted Class	
		+	−
Actual	+	100	0
Class	−	100	900

Since half of the positive predictions made by the classifier are incorrect, it has a FDR value of $100/(100 + 100) = 0.5$. However, its FPR is equal to $100/(100 + 900) = 0.1$, which is quite low. This example shows that in the presence of high skew (i.e., very small value of α), even a small FPR can result in high FDR. See Section 10.6 for further discussion of this issue.

Note that the evaluation measures defined above provide an incomplete representation of performance, because they either only capture the effect of

false positives (e.g., FPR and precision) or the effect of false negatives (e.g., TPR or recall), but not both. Hence, if we optimize only one of these evaluation measures, we may end up with a classifier that shows low FN but high FP, or vice-versa. For example, a classifier that declares every instance to be positive will have a perfect recall, but high FPR and very poor precision. On the other hand, a classifier that is very conservative in classifying an instance as positive (to reduce FP) may end up having high precision but very poor recall. We thus need evaluation measures that account for both types of misclassifications, FP and FN. Some examples of such evaluation measures are summarized by the following definitions.

$$\begin{aligned} \text{Positive Likelihood Ratio} &= \frac{\text{TPR}}{\text{FPR}}. \\ F_1 \text{ measure} &= \frac{2rp}{r+p} = \frac{2 \times \text{TP}}{2 \times \text{TP} + \text{FP} + \text{FN}}. \\ G \text{ measure} &= \sqrt{rp} = \frac{\text{TP}}{\sqrt{(\text{TP}+\text{FP})(\text{TP}+\text{FN})}}. \end{aligned}$$

While some of these evaluation measures are invariant to the skew (e.g., the positive likelihood ratio), others (e.g., precision and the F_1 measure) are sensitive to skew. Further, different evaluation measures capture the effects of different types of misclassification errors in various ways. For example, the F_1 measure represents a harmonic mean between recall and precision, i.e.,

$$F_1 = \frac{2}{\frac{1}{r} + \frac{1}{p}}.$$

Because the harmonic mean of two numbers tends to be closer to the smaller of the two numbers, a high value of F_1-measure ensures that both precision and recall are reasonably high. Similarly, the G measure represents the geometric mean between recall and precision. A comparison among harmonic, geometric, and arithmetic means is given in the next example.

Example 6.9. Consider two positive numbers $a = 1$ and $b = 5$. Their arithmetic mean is $\mu_a = (a + b)/2 = 3$ and their geometric mean is $\mu_g = \sqrt{ab} = 2.236$. Their harmonic mean is $\mu_h = (2 \times 1 \times 5)/6 = 1.667$, which is closer to the smaller value between a and b than the arithmetic and geometric means. ■

A generic extension of the F_1 measure is the F_β measure, which can be defined as follows.

$$F_\beta = \frac{(\beta^2+1)rp}{r+\beta^2 p} = \frac{(\beta^2+1)\times \text{TP}}{(\beta^2+1)\text{TP}+\beta^2\text{FP}+\text{FN}}. \tag{6.106}$$

Both precision and recall can be viewed as special cases of F_β by setting $\beta = 0$ and $\beta = \infty$, respectively. Low values of β make F_β closer to precision, and high values make it closer to recall.

A more general measure that captures F_β as well as accuracy is the weighted accuracy measure, which is defined by the following equation:

$$\text{Weighted accuracy} = \frac{w_1\text{TP}+w_4 TN}{w_1\text{TP}+w_2\text{FP}+w_3\text{FN}+w_4\text{TN}}. \tag{6.107}$$

The relationship between weighted accuracy and other performance measures is summarized in the following table:

Measure	w_1	w_2	w_3	w_4
Recall	1	1	0	0
Precision	1	0	1	0
F_β	β^2+1	β^2	1	0
Accuracy	1	1	1	1

6.11.3 Finding an Optimal Score Threshold

Given a suitably chosen evaluation measure E and a distribution of classification scores, $s(\mathbf{x})$, on a validation set, we can obtain the optimal score threshold s^* on the validation set using the following approach:

1. Sort the scores in increasing order of their values.
2. For every unique value of score, s, consider the classification model that assigns an instance $\mathbf{x}$ as positive only if $s(\mathbf{x}) > s$. Let $E(s)$ denote the performance of this model on the validation set.
3. Find s^* that maximizes the evaluation measure, $E(s)$.

$$s^* = \text{argmax}_s\ E(s).$$

Note that s^* can be treated as a hyper-parameter of the classification algorithm that is learned during model selection. Using s^*, we can assign a

positive label to a future test instance $\mathbf{x}$ only if $s(\mathbf{x}) > s^*$. If the evaluation measure E is skew invariant (e.g., Positive Likelihood Ratio), then we can select s^* without knowing the skew of the test set, and the resultant classifier formed using s^* can be expected to show optimal performance on the test set (with respect to the evaluation measure E). On the other hand, if E is sensitive to the skew (e.g., precision or F_1-measure), then we need to ensure that the skew of the validation set used for selecting s^* is similar to that of the test set, so that the classifier formed using s^* shows optimal test performance with respect to E. Alternatively, given an estimate of the skew of the test data, α, we can use it along with the TPR and TNR on the validation set to estimate all entries of the confusion matrix (see Table 6.7), and thus the estimate of any evaluation measure E on the test set. The score threshold s^* selected using this estimate of E can then be expected to produce optimal test performance with respect to E. Furthermore, the methodology of selecting s^* on the validation set can help in comparing the test performance of different classification algorithms, by using the optimal values of s^* for each algorithm.

6.11.4 Aggregate Evaluation of Performance

Although the above approach helps in finding a score threshold s^* that provides optimal performance with respect to a desired evaluation measure and a certain amount of skew, α, sometimes we are interested in evaluating the performance of a classifier on a number of possible score thresholds, each corresponding to a different choice of evaluation measure and skew value. Assessing the performance of a classifier over a range of score thresholds is called **aggregate evaluation** of performance. In this style of analysis, the emphasis is not on evaluating the performance of a single classifier corresponding to the optimal score threshold, but to assess the general quality of *ranking* produced by the classification scores on the test set. In general, this helps in obtaining robust estimates of classification performance that are not sensitive to specific choices of score thresholds.

ROC Curve

One of the widely-used tools for aggregate evaluation is the **receiver operating characteristic** (ROC) curve. An ROC curve is a graphical approach for displaying the trade-off between TPR and FPR of a classifier, over varying score thresholds. In an ROC curve, the TPR is plotted along the y-axis and the FPR is shown on the x-axis. Each point along the curve corresponds to a

classification model generated by placing a threshold on the test scores produced by the classifier. The following procedure describes the generic approach for computing an ROC curve:

1. Sort the test instances in increasing order of their scores.
2. Select the lowest ranked test instance (i.e., the instance with lowest score). Assign the selected instance and those ranked above it to the positive class. This approach is equivalent to classifying all the test instances as positive class. Because all the positive examples are classified correctly and the negative examples are misclassified, TPR = FPR = 1.
3. Select the next test instance from the sorted list. Classify the selected instance and those ranked above it as positive, while those ranked below it as negative. Update the counts of TP and FP by examining the actual class label of the selected instance. If this instance belongs to the positive class, the TP count is decremented and the FP count remains the same as before. If the instance belongs to the negative class, the FP count is decremented and TP count remains the same as before.
4. Repeat Step 3 and update the TP and FP counts accordingly until the highest ranked test instance is selected. At this final threshold, TPR = FPR = 0, as all instances are labeled as negative.
5. Plot the TPR against FPR of the classifier.

Example 6.10. [Generating ROC Curve] Figure 6.52 shows an example of how to compute the TPR and FPR values for every choice of score threshold. There are five positive examples and five negative examples in the test set. The class labels of the test instances are shown in the first row of the table, while the second row corresponds to the sorted score values for each instance. The next six rows contain the counts of TP, FP, TN, and FN, along with their corresponding TPR and FPR. The table is then filled from left to right. Initially, all the instances are predicted to be positive. Thus, $TP = FP = 5$ and TPR = FPR = 1. Next, we assign the test instance with the lowest score as the negative class. Because the selected instance is actually a positive example, the TP count decreases from 5 to 4 and the FP count is the same as before. The FPR and TPR are updated accordingly. This process is repeated until we reach the end of the list, where TPR = 0 and FPR=0. The ROC curve for this example is shown in Figure 6.53.

■

Class	+	−	+	−	−	−	+	−	+	+	
	0.25	0.43	0.53	0.76	0.85	0.85	0.85	0.87	0.93	0.95	1.00
TP	5	4	4	3	3	3	3	2	2	1	0
FP	5	5	4	4	3	2	1	1	0	0	0
TN	0	0	1	1	2	3	4	4	5	5	5
FN	0	1	1	2	2	2	2	3	3	4	5
TPR	1	0.8	0.8	0.6	0.6	0.6	0.6	0.4	0.4	0.2	0
FPR	1	1	0.8	0.8	0.6	0.4	0.2	0.2	0	0	0

Figure 6.52. Computing the TPR and FPR at every score threshold.

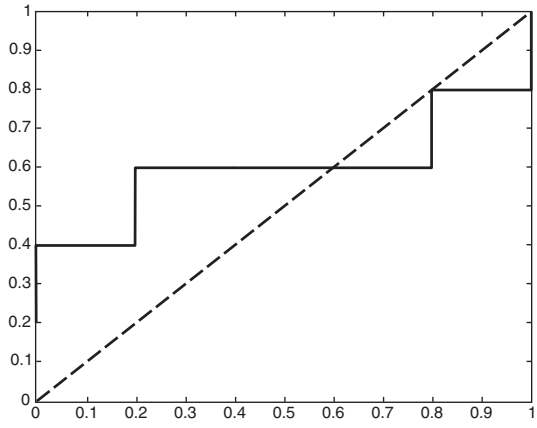


Figure 6.53. ROC curve for the data shown in Figure 6.52.

Note that in an ROC curve, the TPR monotonically increases with FPR, because the inclusion of a test instance in the set of predicted positives can either increase the TPR or the FPR. The ROC curve thus has a staircase pattern. Furthermore, there are several critical points along an ROC curve that have well-known interpretations:

(TPR=0, FPR=0): Model predicts every instance to be a negative class.
(TPR=1, FPR=1): Model predicts every instance to be a positive class.
(TPR=1, FPR=0): The *perfect* model with zero misclassifications.

A good classification model should be located as close as possible to the upper left corner of the diagram, while a model that makes random guesses should reside along the main diagonal, connecting the points ($TPR = 0, FPR = 0$) and ($TPR = 1, FPR = 1$). Random guessing means that an

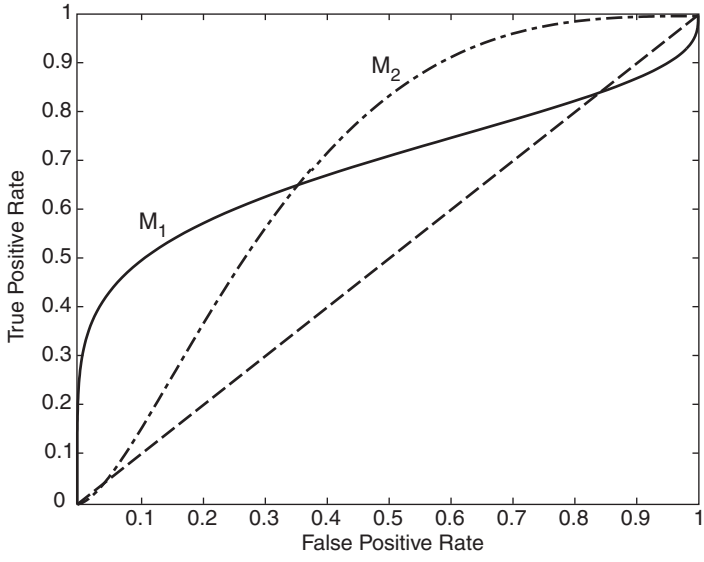


Figure 6.54. ROC curves for two different classifiers.

instance is classified as a positive class with a fixed probability p, irrespective of its attribute set. For example, consider a data set that contains n_+ positive instances and n_- negative instances. The random classifier is expected to correctly classify pn_+ of the positive instances and to misclassify pn_- of the negative instances. Therefore, the TPR of the classifier is $(pn_+)/n_+ = p$, while its FPR is $(pn_-)/p = p$. Hence, this random classifier will reside at the point (p, p) in the ROC curve along the main diagonal.

Since every point on the ROC curve represents the performance of a classifier generated using a particular score threshold, they can be viewed as different **operating points** of the classifier. One may choose one of these operating points depending on the requirements of the application. Hence, an ROC curve facilitates the comparison of classifiers over a range of operating points. For example, Figure 6.54 compares the ROC curves of two classifiers, M_1 and M_2, generated by varying the score thresholds. We can see that M_1 is better than M_2 when FPR is less than 0.36, as M_1 shows better TPR than M_2 for this range of operating points. On the other hand, M_2 is superior when FPR is greater than 0.36, since the TPR of M_2 is higher than that of M_1 for this range. Clearly, neither of the two classifiers **dominates** (is strictly better than) the other, i.e., shows higher values of TPR and lower values of FPR over all operating points.

To summarize the aggregate behavior across all operating points, one of the commonly used measures is the **area under the ROC curve** (AUC). If the classifier is perfect, then its area under the ROC curve will be equal 1. If the algorithm simply performs random guessing, then its area under the ROC curve will be equal to 0.5.

Although the AUC provides a useful summary of aggregate performance, there are certain caveats in using the AUC for comparing classifiers. First, even if the AUC of algorithm A is higher than the AUC of another algorithm B, this does not mean that algorithm A is always better than B, i.e., the ROC curve of A dominates that of B across all operating points. For example, even though M_1 shows a slightly lower AUC than M_2 in Figure 6.54, we can see that both M_1 and M_2 are useful over different ranges of operating points and none of them are strictly better than the other across all possible operating points. Hence, we cannot use the AUC to determine which algorithm is better, unless we know that the ROC curve of one of the algorithms dominates the other.

Second, although the AUC summarizes the aggregate performance over all operating points, we are often interested in only a small range of operating points in most applications. For example, even though M_1 shows slightly lower AUC than M_2, it shows higher TPR values than M_2 for small FPR values (smaller than 0.36). In the presence of class imbalance, the behavior of an algorithm over small FPR values (also termed as **early retrieval**) is often more meaningful for comparison than the performance over all FPR values. This is because, in many applications, it is important to assess the TPR achieved by a classifier in the first few instances with highest scores, without incurring a large FPR. Hence, in Figure 6.54, due to the high TPR values of M_1 during early retrieval (FPR < 0.36), we may prefer M_1 over M_2 for imbalanced test sets, despite the lower AUC of M_1. Hence, care must be taken while comparing the AUC values of different classifiers, usually by visualizing their ROC curves rather than just reporting their AUC.

A key characteristic of ROC curves is that they are agnostic to the skew in the test set, because both the evaluation measures used in constructing ROC curves (TPR and FPR) are invariant to class imbalance. Hence, ROC curves are not suitable for measuring the impact of skew on classification performance. In particular, we will obtain the same ROC curve for two test data sets that have very different skew.

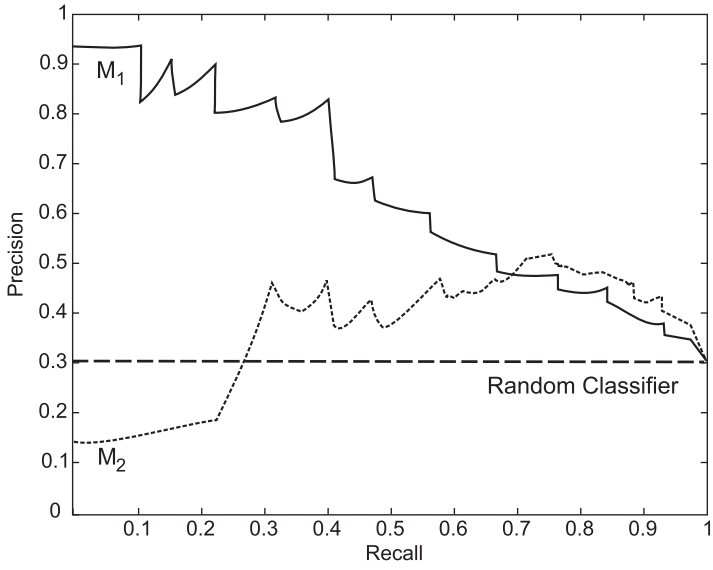


Figure 6.55. PR curves for two different classifiers.

Precision-Recall Curve

An alternate tool for aggregate evaluation is the **precision recall curve** (PR curve). The PR curve plots the precision and recall values of a classifier on the y and x axes respectively, by varying the threshold on the test scores. Figure 6.55 shows an example of PR curves for two hypothetical classifiers, M_1 and M_2. The approach for generating a PR curve is similar to the approach described above for generating an ROC curve. However, there are some key distinguishing features in the PR curve:

1. PR curves are sensitive to the skew factor $\alpha = P/(P+N)$, and different PR curves are generated for different values of α.

2. When the score threshold is lowest (every instance is labeled as positive), the precision is equal to α while recall is 1. As we increase the score threshold, the number of predicted positives can stay the same or decrease. Hence, the recall monotonically declines as the score threshold increases. In general, the precision may increase or decrease for the same value of recall, upon addition of an instance into the set of predicted positives. For example, if the kth ranked instance belongs to the negative class, then including it will result in a drop in the precision without affecting the recall. The precision may improve at the next step, which

adds the $(k+1)^{\text{th}}$ ranked instance, if this instance belongs to the positive class. Hence, the PR curve is not a smooth, monotonically increasing curve like the ROC curve, and generally has a zigzag pattern. This pattern is more prominent in the left part of the curve, where even a small change in the number of false positives can cause a large change in precision.

3. As, as we increase the imbalance among the classes (reduce the value of α), the rightmost points of all PR curves will move downwards. At and near the leftmost point on the PR curve (corresponding to larger values of score threshold), the recall is close to zero, while the precision is equal to the fraction of positives in the top ranked instances of the algorithm. Hence, different classifiers can have different values of precision at the leftmost points of the PR curve. Also, if the classification score of an algorithm monotonically varies with the posterior probability of the positive class, we can expect the PR curve to gradually decrease from a high value of precision on the leftmost point to a constant value of α at the rightmost point, albeit with some ups and downs. This can be observed in the PR curve of algorithm M_1 in Figure 6.55, which starts from a higher value of precision on the left that gradually decreases as we move towards the right. On the other hand, the PR curve of algorithm M_2 starts from a lower value of precision on the left and shows more drastic ups and downs as we move right, suggesting that the classification score of M_2 shows a weaker monotonic relationship with the posterior probability of the positive class.

4. A random classifier that assigns an instance to be positive with a fixed probability p has a precision of α and a recall of p. Hence, a classifier that performs random guessing has a horizontal PR curve with $y = \alpha$, as shown using a dashed line in Figure 6.55. Note that the random baseline in PR curves depends on the skew in the test set, in contrast to the fixed main diagonal of ROC curves that represents random classifiers.

5. Note that the precision of an algorithm is impacted more strongly by false positives in the top ranked test instances than the FPR of the algorithm. For this reason, the PR curve generally helps to magnify the differences between classifiers in the left portion of the PR curve. Hence, in the presence of class imbalance in the test data, analyzing the PR curves generally provides a deeper insight into the performance of classifiers than the ROC curves, especially in the early retrieval range of operating points.

6. The classifier corresponding to (precision = 1, recall = 1) represents the perfect classifier. Similar to AUC, we can also compute the area under the PR curve of an algorithm, known as AUC-PR. The AUC-PR of a random classifier is equal to α, while that of a perfect algorithm is equal to 1. Note that AUC-PR varies with changing skew in the test set, in contrast to the area under the ROC curve, which is insensitive to the skew. The AUC-PR helps in accentuating the differences between classification algorithms in the early retrieval range of operating points. Hence, it is more suited for evaluating classification performance in the presence of class imbalance than the area under the ROC curve. However, similar to ROC curves, a higher value of AUC-PR does not guarantee the superiority of a classification algorithm over another. This is because the PR curves of two algorithms can easily cross each other, such that they both show better performances in different ranges of operating points. Hence, it is important to visualize the PR curves before comparing their AUC-PR values, in order to ensure a meaningful evaluation.

6.12 Multiclass Problem

Some of the classification techniques described in this chapter are originally designed for binary classification problems. Yet there are many real-world problems, such as character recognition, face identification, and text classification, where the input data is divided into more than two categories. This section presents several approaches for extending the binary classifiers to handle multiclass problems. To illustrate these approaches, let $Y = \{y_1, y_2, \ldots, y_K\}$ be the set of classes of the input data.

The first approach decomposes the multiclass problem into K binary problems. For each class $y_i \in Y$, a binary problem is created where all instances that belong to y_i are considered positive examples, while the remaining instances are considered negative examples. A binary classifier is then constructed to separate instances of class y_i from the rest of the classes. This is known as the one-against-rest (1-r) approach.

The second approach, which is known as the one-against-one (1-1) approach, constructs $K(K-1)/2$ binary classifiers, where each classifier is used to distinguish between a pair of classes, (y_i, y_j). Instances that do not belong to either y_i or y_j are ignored when constructing the binary classifier for (y_i, y_j). In both 1-r and 1-1 approaches, a test instance is classified by combining the predictions made by the binary classifiers. A voting scheme is typically